MEMORA

A MINOR PROJECT REPORT

submitted by

Adithyan P S

KH.SC.I5MCA23015

Under the supervision of

M Soumya Krishnan

Asst. Professor

(Sr. Gr)

Department of Computer Science and IT

in partial fulfilment of the requirement of

AMRITA VISHWA VIDYAPEETHAM

for the award of the degree of

FIVE YEAR INTEGRATED MCA

AMRITA VISHWA VIDYAPEETHAM, KOCHI CAMPUS

November 2025

AMRITA VISHWA VIDYAPEETHAM

AMRITA VISHWA VIDYAPEETHAM, KOCHI CAMPUS BONAFIDE CERTIFICATE

This is to certify that the minor project report entitled **SMART REMINDER AND TASK MANAGEMENT APP FOR ADHD AND EARLY DEMENTIA(MEMORA)** submitted by **ADITHYAN P S - (KH.SC.I5MCA23015)** in partial fulfilment of the requirements for the award of the **INTEGRATED MASTERS OF COMPUTER APPLICATIONS** is a bonafide record of the work carried out under my guidance and supervision at School of Computing, Amrita Vishwa Vidyapeetham, Kochi Campus.

M Soumya Krishnan
Project Guide
Asst. Professor (Sr. Gr)
Department of Computer Science
School of Computing
Amrita Vishwa Vidyapeetham
Kochi Campus

Dr. Sangeetha J
Head of the Department
Department of Computer Science
School of Computing
Amrita Vishwa Vidyapeetham
Kochi Campus

The project was evaluated by as on: _____

Internal Examiner **External Examiner**

DECLARATION

I affirm that the minor project work entitled **"Memora"** being submitted in partial fulfilment for the award of the **DEGREE INTEGRATED MASTERS OF COMPUTER APPLICATIONS** is the original work carried out by me. It has not formed the part of any other project work/internship submitted for award of any degree or diploma, either in this or any other University.

Place: Kochi

Date: **Adithyan P S – KH.SC.I5MCA23015**

DEDICATION

To

My parents, all my teachers and the eternal God.
Thank you for believing in me and encouraging me throughout.

ACKNOWLEDGEMENT

A venture can't be completely by themselves. I take this opportunity to gratefully acknowledge various people who acted as guides along the way.

I offer my humble salutations at the lotus feet of holy mother **Sri Mata Amrithananthamayi Devi,** who is guiding light of my life without which I would not have completed my project.

The success of any work requires the blessings of the Lord Almighty. I thank my God for aiding me in my travel to success.

I express my thankfulness to **Dr. U. Krishnakumar**, Director, Amrita Vishwa Vidyapeetham, Kochi Campus for giving me an opportunity to do my project.

I would like to express my deep gratitude to **Dr. Sangeetha J**, Head of the Department, Department of Computer Science, School of Computing, Amrita Vishwa Vidyapeetham, Kochi Campus for her valuable guidance and for the support she rendered on me.

I would also like to thank my guide **M Soumya Krishnan**, Asst.Professor (Sr. Gr) who revised my ideas and helped me to bring it to the Minor Project.

I would also like to express my gratitude to the staff of the ICTS Department, School of Physical Sciences, Amrita Vishwa Vidyapeetham, Kochi Campus for the technical support they gave.

This thanks giving cannot be complete without mentioning my friends and parents who gave the mental strength which I almost lost in between the journey.

Adithyan P S– KH.SC.I5MCA23015

Contents

Abstract

Memora is an Android-based smart reminder and task management application designed to assist individuals with ADHD and early-stage dementia in organizing daily activities and improving memory retention. The app focuses on providing a distraction-free, accessible environment through clear interfaces, personalized reminders, and real-time caregiver collaboration.

The system supports two major user roles — patients and caregivers. ADHD users can independently manage their schedules, while dementia patients receive assistance from linked caregivers who can remotely create, edit, and monitor tasks. A passwordless registration system ensures easy onboarding, followed by an in-app screening test that classifies users based on their cognitive profile, ensuring appropriate access and reducing misuse.

Memora is structured into several functional modules. The Task Management Module handles creating, updating, and completing tasks. The Reminder and Notification Module provide visual and auditory cues via Android's alarm and notification APIs. The Caregiver Module allows caregivers to link with patients, manage notes, and review weekly progress. The Chatbot Module offers basic conversational assistance using keyword recognition and voice input powered by the KPI API, ensuring hands-free accessibility. All user data, including task records and test results, are securely stored in Firebase Firestore8.

Developed using Java and XML in Android Studio, Memora follows an Incremental Development Model with Agile adaptation, allowing iterative updates and continuous testing after each feature release. This flexible workflow enabled frequent refinements based on real usage and feedback. The result is a lightweight, reliable, and privacy-conscious system that enhances user independence while strengthening caregiver support.

By combining intuitive design, cognitive-specific workflows, and cloud-based synchronization, Memora stands as a practical, no-cost digital aid for improving task adherence and daily functioning among individuals with attention and memory challenges.

1.0 INTRODUCTION

1.1 About the System

The *Smart Reminder System – Memora* is a thoughtfully designed mobile application aimed at assisting individuals diagnosed with Attention Deficit Hyperactivity Disorder (ADHD) or early-stage dementia in managing their everyday routines with confidence and independence. These individuals often struggle with remembering essential activities such as taking medication on time, eating regular meals, attending appointments, or maintaining personal hygiene. Memora addresses these challenges through a structured yet compassionate approach by offering smart, repeatable, and easy-to-understand reminders that fit seamlessly into the user's lifestyle.

The application focuses on accessibility and simplicity. It uses a clean and intuitive interface with large, clearly labelled buttons, visually distinct icons, and voice chat to ensure that users can navigate it effortlessly. The system integrates AI-driven assistance by chat bot to reminders and weekly interaction history. Its voice-text capabilities make it especially useful for users who may find text input difficult.

Beyond serving as an academic project, *Memora* embodies a meaningful real-world purpose, to create an inclusive technological bridge between clinical recommendations and practical daily application. For individuals with ADHD, the app provides structured motivation through engaging reminders and color-coded priority-based task screens that sustain attention. For dementia patients, it offers calm notifications, simplified menus, and caregiver integration, reducing confusion while promoting independence. Caregivers, in turn, can remotely assign or monitor tasks, receive alerts for missed activities, and access progress reports, thereby lowering stress and ensuring timely interventions.

By combining affordability, AI chat bot, and empathetic design, *Memora* redefines what a digital support tool can be. It empowers users to lead more organized, self-reliant lives and gives caregivers peace of mind knowing that their loved ones are safely guided through each day. Ultimately, *Memora* is not just a reminder app it is a step toward accessible digital healthcare, merging innovation with compassion to improve the quality of life for those who need it most.

2.0 NEED FOR THE SYSTEM

Cognitive disorders such as Attention Deficit Hyperactivity Disorder (ADHD) and dementia significantly affect an individual's ability to manage daily tasks, maintain consistency in routines, and preserve independence. Globally, more than 57 million people live with dementia, and approximately 10 million new cases are reported every year (WHO, 2021). In India alone, around 8.8 million individuals over the age of 60 are affected, with states like Kerala showing higher prevalence due to an ageing population and increasing life expectancy. ADHD, on the other hand, affects about 7–8% of children worldwide, with nearly 5–8% of the Indian population experiencing symptoms that often persist into adulthood.

Despite these alarming statistics, both ADHD and dementia remain poorly understood in the public sphere. In many parts of India, these conditions are often misinterpreted as personality flaws, laziness, or simply a natural part of ageing. The lack of awareness and timely intervention results in unmanaged symptoms, dependency on caregivers, and declining quality of life.

Existing reminder and cognitive-support applications fail to address these issues effectively. Most of them are either subscription-based, limiting access for economically weaker users, or designed with complex user interfaces that are unsuitable for people with cognitive impairments. Others lack essential functions like condition-specific reminders, caregiver monitoring, or AI-based personalization — all of which are vital for these user groups. Furthermore, data privacy, affordability, and usability remain major concerns in currently available tools.

The *Smart Reminder System – Memora* bridges these gaps by providing a free, inclusive, and feature-rich solution designed specifically for ADHD and early dementia users. It combines AI-driven smart reminders, accessible design elements, and caregiver integration to ensure that users receive the right support at the right time. By translating medical recommendations into practical, daily assistance, Memora empowers patients to regain confidence and autonomy in their routines, while reducing the emotional and logistical burden on caregivers.

Ultimately, the need for this system stems from a growing societal requirement for affordable digital healthcare tools that are empathetic, accessible, and adaptive to individual cognitive needs. Memora not only fulfills this technological necessity but also contributes meaningfully to social well-being and inclusive healthcare innovation.

3.0 BACKGROUND STUDY

3.1 Existing Systems

Case Study 1: TickTick App

User-Reported:

- Complex UI: Perceived as cluttered; basic actions require multiple steps.
- Technical Glitches: Disappearing subtasks, duplicate/deleted tasks, broken repeats.
- Notification Issues: Missed reminders, random chimes, difficult to disable alerts.
- Feature & Plan Limits: Strict free-tier caps; high premium pricing.
- Sync Problems: Inconsistent integration with Google Calendar.
- Forced Upgrades: Free-tier feature reductions over time.
- Customization Gaps: Limited repeat options; unclear auto start setup.
- Feature Overload: Extra tools (Pomodoro, habit tracking) may clutter workflow.

Security & Privacy Concerns:

- Data hosted on US-based AWS servers; no end-to-end encryption; unclear data retention policies.

Workflow Limitations:

- Homepage is calendar view which is very disorientated and gives a complex overview, undo limitations, and cluttered extra tools.
- No Caregiver Help: The dementia patient has to set tasks on their own which is difficult for them.
- Cost Barrier: Several essential features locked behind a premium subscription, limiting access for low-income users.

Case Study 2: Finch App

User Reported:

- High Subscription Cost: Most advanced and essential features are restricted to paid subscription plans, limiting free users from accessing important tools that could support their mental well-being.
- Anxiety Triggers: Frequent notifications, countdown timers, and goal streaks can unintentionally increase stress and anxiety levels among users, especially those with attention or anxiety disorders.

- Not Patient Specific: The app is built for general self-improvement rather than being tailored to individuals with conditions like ADHD or dementia, resulting in a less personalized and less effective user experience.
- Mis-usable: The app does not perform any initial screening or verification to determine if users actually have a mental or emotional disorder, making it prone to misuse or irrelevant usage.
- Dependency Risk**:** Users may develop an overreliance on the app for emotional regulation, mistaking it as a substitute for professional mental health care or therapy.
- Motivation Challenges: The reward system (points, virtual pets, etc.) may not always correspond to meaningful task completion, causing users to focus on superficial achievements instead of actual personal growth.
- Cost Barrier: Many core and helpful features are locked behind a paywall, making it difficult for users who cannot afford premium plans to access consistent mental health support.

Security & Privacy Concerns:

- Data Handling Risks: The app collects sensitive emotional and behavioural data but lacks clarity on how long data is stored or how securely it is managed. End-to-end encryption is not explicitly confirmed, raising potential privacy concerns.

Workflow Limitations:

- Overstimulating Interface**:** The app's colourful and interactive design may overwhelm users with cognitive or attention difficulties, reducing usability for target audiences.
- No Caregiver Support: Finch does not include caregiver integration or shared monitoring options, which limits its usefulness for patients who need guided assistance (e.g., dementia users).
- Cost Barrier: Similar to many self-care apps, vital mental health support tools and insights are restricted to premium users, creating accessibility challenges for low-income users.

3.2 Drawbacks of Existing Systems

The analysis of the existing systems reveals several critical drawbacks that significantly limit their usability, accessibility, and overall effectiveness for end-users. These issues not only affect user experience but also influence motivation, data security, and emotional well-being. The detailed elaboration of these drawbacks is as follows:

1. Complex User Interface:

Current task management and mental wellness applications, such as Tick Tick, often feature cluttered and overly complicated designs. Simple actions like creating, editing, or completing a task may require navigating through multiple screens or nested menus. This results in a steep learning curve for new users and increases the time taken to perform routine tasks. Moreover, the abundance of icons, options, and animations often distracts users from their primary purpose—managing their tasks efficiently**.**

2. Technical Instability:

Many users have reported frequent technical glitches, including disappearing subtasks, duplicated entries, and malfunctioning reminders. These issues not only disrupt workflow continuity but also reduce the reliability of the system. When an application fails to save or recall crucial user data accurately, it erodes trust and discourages long-term usage. The instability of sync mechanisms and update bugs further add to this inconsistency.

3. Limited Free Features:

The free versions of most productivity applications provide only minimal functionality, restricting access to essential tools like recurring tasks, advanced reminders, or collaborative features. Users are often forced to upgrade to premium plans, which come with high subscription costs. This pricing structure creates a financial barrier, particularly for students, low-income individuals, or those seeking simple productivity aids without recurring payments.

4. Sync and Integration Issues:

Inconsistent synchronization with third-party services such as Google Calendar, Outlook, or Apple Reminders leads to missed events, double entries, and scheduling errors. These integration issues cause confusion when users rely on multiple platforms to organize their daily lives. Delayed or incomplete sync also results in a loss of real-time accuracy, which is critical for productivity-based systems.

5. Overloaded Features:

Several applications attempt to include a wide range of features such as Pomodoro timers, habit trackers, journals, and mood logs—all in one place. While these tools can be useful individually, their unnecessary inclusion often clutters the interface and confuses users. Instead of simplifying productivity, these apps overwhelm users with too many functions that divert attention from the main purpose—effective task and time management.

6. Customization Constraints:

Users often find limited options for personalizing their experience. For example, restrictions in setting flexible task repetitions, custom reminders, or unique notification sounds prevent users from adapting the app to their specific workflow. This lack of customization makes the system feel rigid and impersonal, ultimately reducing user satisfaction and long-term engagement.

7. Security and Privacy Concerns:

Many applications lack strong data protection measures like end-to-end encryption, leaving user information vulnerable. Unclear data policies regarding storage and sharing further reduce user trust, especially in apps handling sensitive personal data.

8. Emotional and Psychological Strain:

Apps like finch and similar motivational platforms can unintentionally cause stress or anxiety. Frequent notifications, countdown timers, and performance-based reward systems can create pressure rather than encouragement. Users may begin to associate productivity with guilt or failure instead of positive reinforcement, negatively impacting mental well-being.

9. Dependency and Motivation Issues:

Over time, users may develop dependency on external validation provided by app-based systems such as streaks, badges, or progress bars. This dependence can reduce intrinsic motivation—users may struggle to perform tasks independently without the app's guidance or rewards. Consequently, while short-term motivation may increase, long-term self-discipline and internal motivation may decline.

10. Accessibility and Affordability Barriers:

Many applications fail to address inclusivity by placing essential or advanced features behind paywalls. This restricts access for users from lower economic backgrounds or those unwilling to commit to subscriptions. Additionally, the lack of accessibility features such as voice commands, screen reader support, or simplified modes makes it difficult for differently-abled users to benefit from these syst

3.3 Proposed System

The proposed Smart Reminder system is designed to directly address the challenges faced by individuals with ADHD and early-stage Dementia, while also supporting their caregivers in managing daily routines. The system is built on the principle of simplicity with intelligence, delivering essential reminder and tracking features in an interface that is easy to navigate, even for users with cognitive impairments.

In the beginning, the app provide a password less login/register for the patients which is very suitable for cognitive people with possible memory loss. Further the users are faced with a screening test interface before registration to confirm the disorder to avoid misuse of the app filtering people who confuses laziness for ADHD and misusers. The test displays the level of the user of the scale of ‘minimal’ to ‘severe’ and only allows the user to register if he has moderate or higher chances of the disorder else shows an exit button to exit from the app.

At its core, the application allows users to create, manage, and complete tasks with the assistance of visual cues, smart notifications and alarms. The workflow for ADHD and dementia patients differs to suit their distinct needs: ADHD patient has the freedom to set their tasks for their day which gives them the sense of responsibility, while there is caregiver (a physical caretaker or family member) to manage the tasks of dementia patients. The users receive vibrant, engaging notifications to sustain focus and motivation and a simplified interface to reduce confusion. Caregivers have access to their own interface, enabling them to remotely add, edit, or monitor tasks, ensuring that critical activities are not missed. The task page classifies task as current and upcoming, while also separating complete and incomplete tasks. The page is also refreshed daily removing past task to reduce clutter or confusion.

The app gives weekly and daily reports with weekly history view to see all the task assigned for that week and provides a completion rate for each week. The integration of an AI assistant enhances

the system's value by providing voice-chat, doubt solving, daily briefings and guidance through the app.

Unlike most competing applications, Memora offers all its core features free of charge, eliminating financial barriers and ensuring accessibility to users across income levels. This combination of affordability, personalization, and caregiver integration positions Memora as a unique and impactful solution in the cognitive support tools market.

4.0 PROBLEM FORMULATION

Individuals with ADHD and early-stage dementia face daily challenges in organizing tasks, maintaining focus, and remembering essential routines such as taking medication, attending appointments, and completing daily activities. Existing reminder systems are either overly complex, lack cognitive-specific features, or require paid subscriptions, making them inaccessible to those who need them most.

The Smart Reminder (Memora) project aims to formulate a digital solution that is both intelligent and inclusive, providing patient specific reminder systems with notifications and alarms, caregiver collaboration, daily report tracking to help users manage their day effectively and independently.

4.1 Main Objectives

- To design and develop a **mobile-based reminder application** tailored for individuals with ADHD and early-stage dementia.
- To offer **reminders and smart task management** with support for text-based cues.
- To provide **caregiver integration**, enabling real-time task monitoring and remote assistance.
- To ensure **AI-driven adaptability** through voice commands, enquiry knowledge about the disorder, provides guidelines to use the app properly.
- To maintain **data security and accessibility**, ensuring safe and reliable cloud-based storage using Firebase.

4.2 Specific objectives

- Develop a **simple and intuitive UI/UX** suitable for users with cognitive impairments.
- Implement a **Patient identification module** that identifies user type (ADHD or dementia or not) during onboarding.
- Integrate **condition-specific workflows** vibrant, stimulating alarms for ADHD users and caregiver guided reminders for dementia users.
- Build a **caregiver dashboard** for dementia patients, that allows caregivers to monitor and keep notes about the patient progress, and manage schedules remotely for the patient.
- Utilize **lightweight AI models or APIs** (e.g., Vosk and firebase APIs) for voice input and

daily briefings.

- Generate **daily and weekly reports** on user progress and provide a weekly history with completion status.
- Optimize the app for **low-end devices** to maximize accessibility.
- Requires **less storage space** required as all data are stored in the cloud storage and is only accessed at real time.

4.3 Methodology

The development process follows an Incremental Development Model with Agile Adaptation, allowing for continuous changes and iteration during the design and testing phases.

Requirement Analysis: Identify real time user needs, caregiver expectations, and technical constraints with feasible features.

System Design: Develop wireframes, UI mockups, and define database structure using Firebase Firestore.

Implementation:

- **Frontend:** XML framework for multi-platform mobile UI.
- **Backend:** Java for API handling and Firebase integration.
- **Database:** Cloud Firestore for real-time synchronization.
- **AI Integration:** Incorporate lightweight AI modules for voice recognition, user guidelines and daily briefing reports.
- **Testing:** Conduct usability, functional, and accessibility testing with a focus on cognitive ease.
- **Deployment:** Deploy the final app via GitHub pages.

4.4 Platform Selection

The platform is chosen based on ease of development, cost efficiency and cloud integration.

Frontend Platform: *XML* – chosen for its fast development, adaptive UI for accessibility, and ability to run on Android.

Backend Platform: *Java* – lightweight and well-suited for quick API development with Firebase support.

Database: *Firebase Firestore* – offers real-time cloud storage, authentication, and secure data handling.

AI Integration Tools: *Vosk API* and other lightweight NLP tools for voice-to-text and command processing.

Development Environment: *Android Studio* for integrated testing, debugging, and deployment.

5.0 SYSTEM ANALYSIS & DESIGN

The system analysis phase focuses on understanding the functional requirements, identifying the problems within existing solutions, and defining how the new system will address these challenges effectively. The design phase transforms these requirements into a structured plan that guides development, ensuring the final product is reliable, efficient, and user-friendly.

5.1 System Analysis

The Smart Reminder (Memora) system is designed after carefully analyzing the behavior and needs of real-life ADHD and early-stage dementia patients. Both user groups experience cognitive delays, attention deficits, and forgetfulness, which require a personalized digital intervention.

Through user research and case studies, major challenges were identified: complex UIs, inconsistent notifications, subscription-based limitations, and lack of caregiver collaboration.

To overcome these, Memora focuses on simplicity, clarity, and adaptability—offering reminders, large interactive elements and real-time caregiver access.

Key Functionalities:

- Reminder scheduling and task categorization
- Test interfaces for each patient, to restrict misuse of our app
- Notifications that notify 5 minutes before reminder time & task completion
- Caregiver-linked task monitoring and management for dementia patients
- AI-driven chat bot which can answer enquiries about the app and diagnosed conditions
- Reports with completion rate on a weekly basis
- Secure data management with Firebase Firestore
- Cloud Synchronization & Remote Access: Real-time updates between patients and caregivers across multiple devices.

This system ensures that users are guided through a structured, stress-free experience that encourages consistency and autonomy.

5.2 Feasibility Analysis

The system can be developed successfully with the available time, resources, and technology. Memora was implemented with **minimal cost** by using open-source tools and APIs such as Firebase Authentication, Android Alarm Manager, and the **VOSK API** (from Alpha Cephei) for speech-to-text functionality. This ensured smooth, affordable, and efficient development.

5.2.1 Technical Analysis

The system uses widely accessible, beginner-friendly, and open-source tools:

- **Frontend:** Developed using **XML**, ensuring responsive design, high accessibility, and compatibility across Android devices.
- **Backend:** Built with **JAVA**, offering lightweight and flexible API management.
- **Database: Firebase Firestore** for real-time, secure, and scalable data storage.
- **AI Integration:** Uses lightweight APIs (e.g., Vosk…) for speech recognition. It chat bot performs daily and weekly briefing.

All technologies are cloud-supported and well-documented, ensuring technical viability even for small development teams.

All technologies are cloud-supported and well-documented, ensuring technical viability even for small development teams

5.2.2 Economical Analysis

The project is highly economical because it relies entirely on **free and open-source tools**.

- No license or subscription fees are required for Android Studio (Java/XML), or Firebase.
- Hosting and authentication services are available through the **Firebase free tier**, reducing operational costs.
- Screening test were developed according to WHO standards. (ASRS for ADHD and MoCA for Dementia)
- Alarm API was from Android Studio, which was inbuilt API
- The system is deployable on any Android smartphone, eliminating hardware dependency.

Hence, the total cost of development remains minimal, making the solution accessible and sustainable for both users and developers.

5.2.3 Performance Analysis

The system's performance goals are centred around **speed, responsiveness, and reliability**.

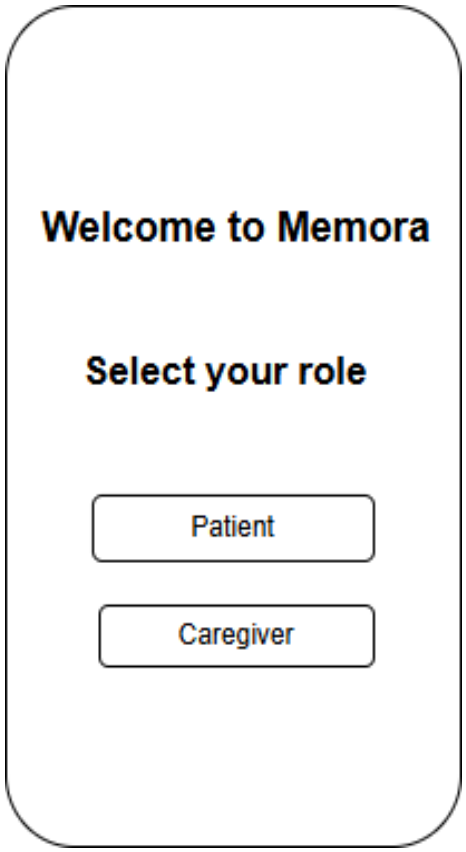
- The app is optimized to run smoothly even on low-end Android devices.
- Firebase ensures real-time synchronization between users and caregivers.
- Background services handle notifications and voice processing efficiently.

Overall, the design ensures that the system delivers high performance with minimal latency, even under variable network conditions.

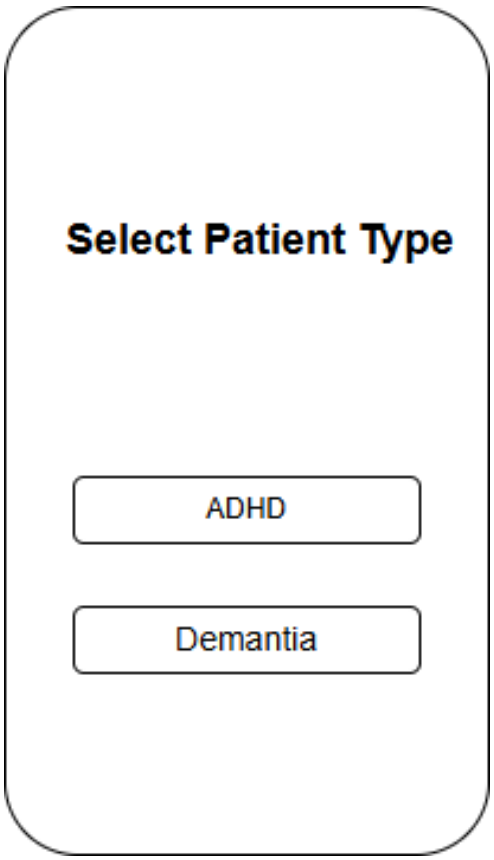
6.0 System Design

6.1 Input Design

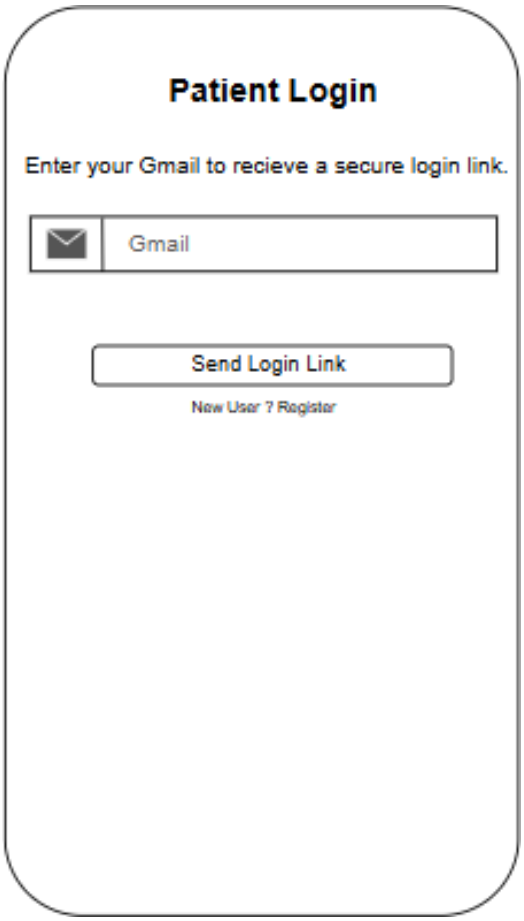
Role selection

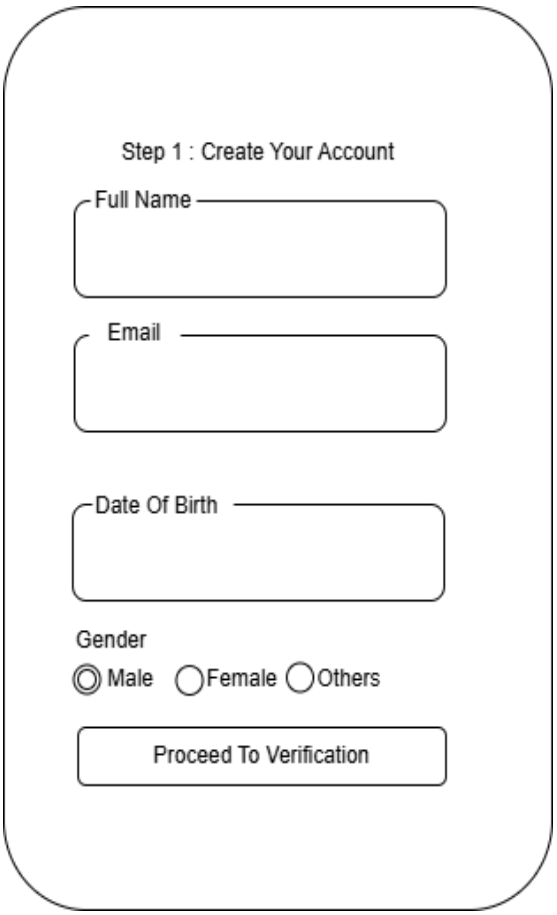


Patient selection

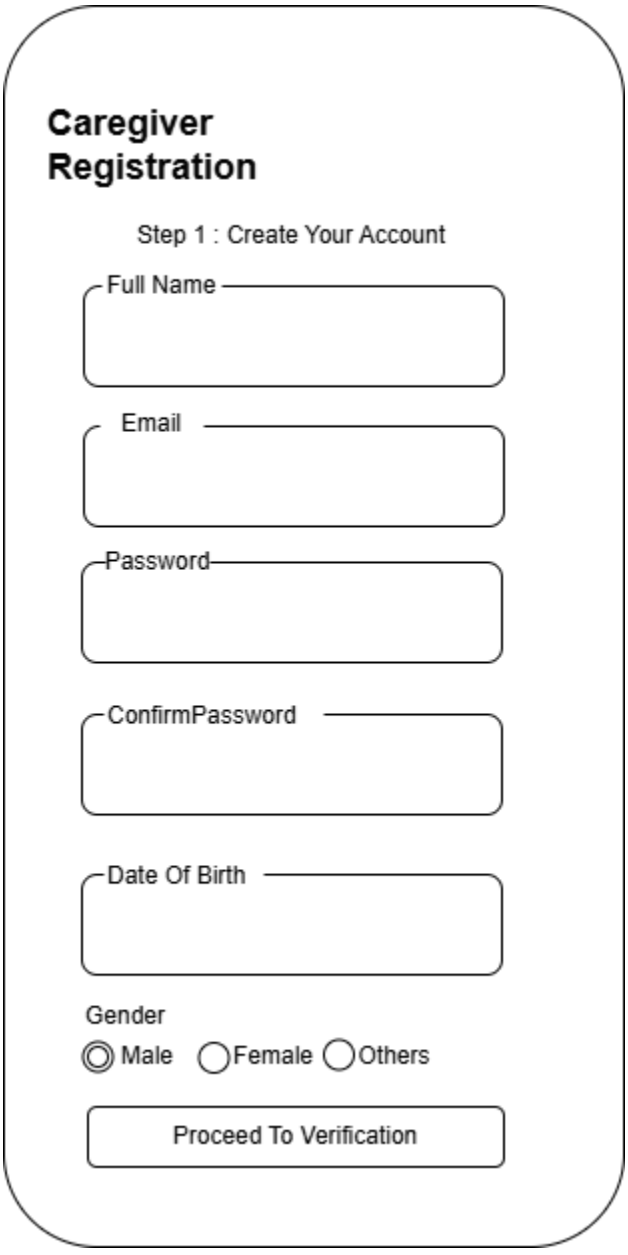


Login and registration

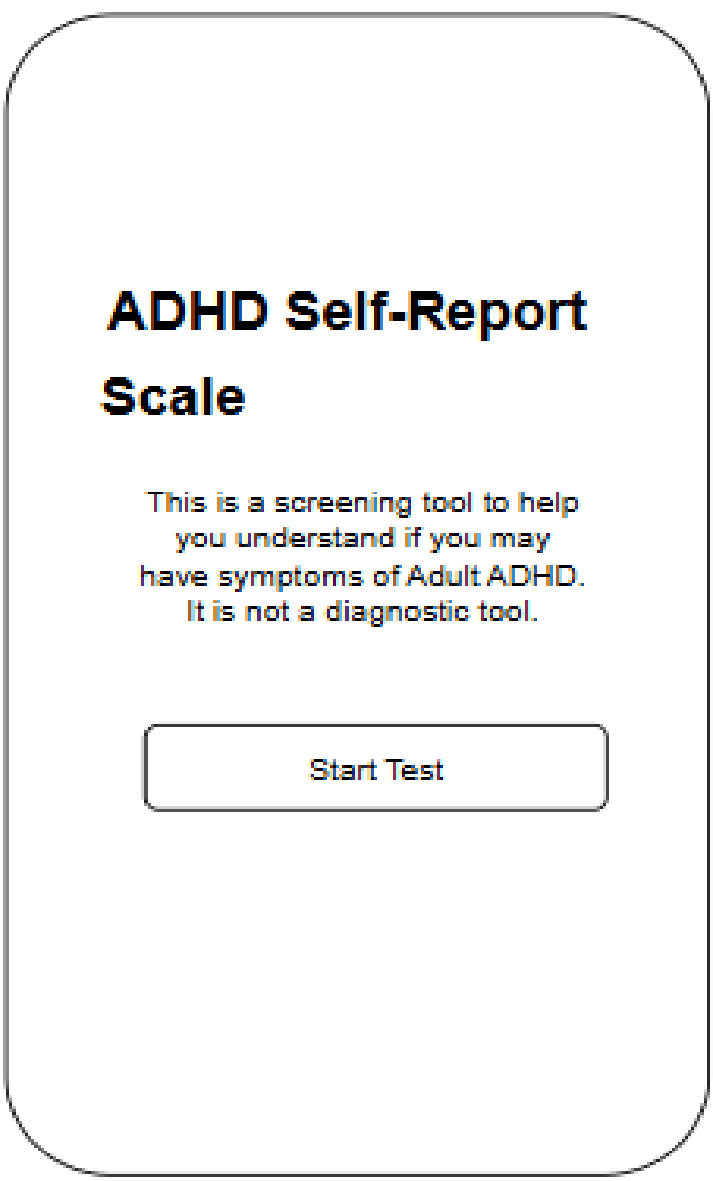


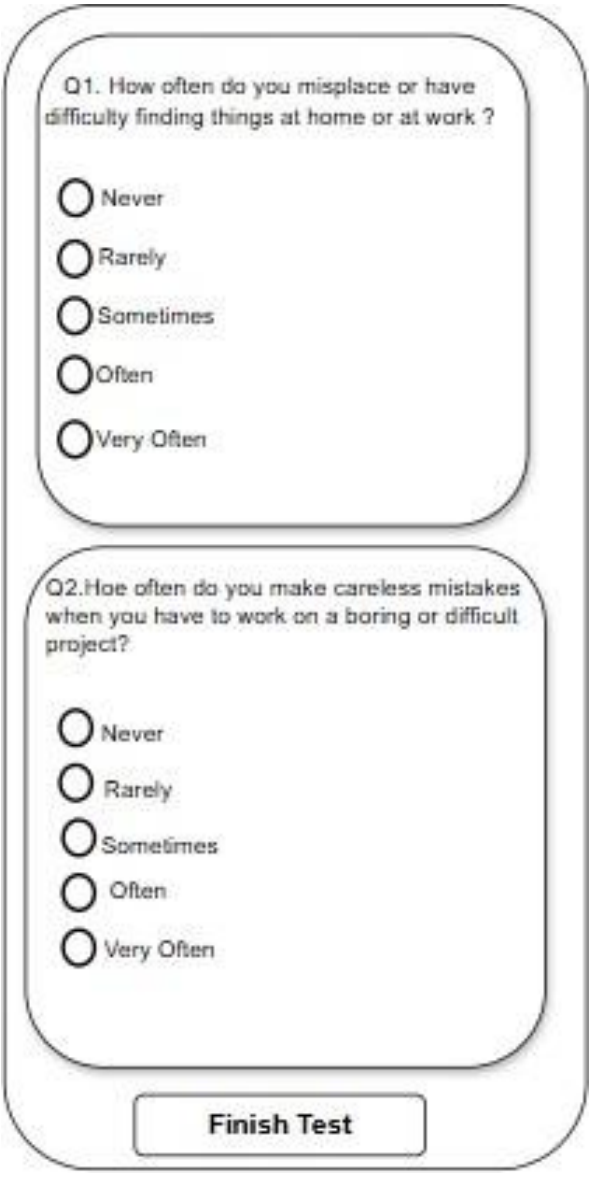


Caregiver registration



ADHD Test Interfaces



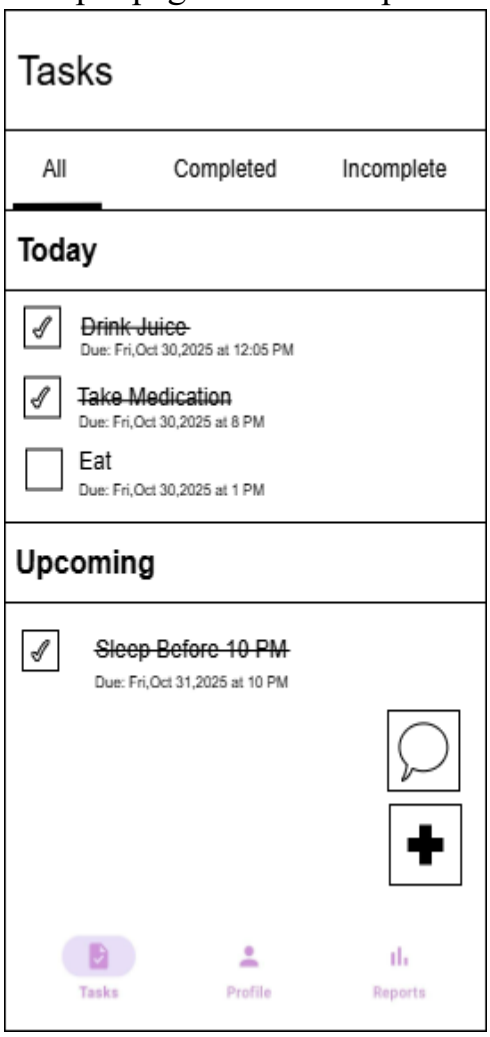


Chat bot


Task Today
I'm sorry,I can't access your schedule without a valid user ID
Type here...
Sent
Speak

Dementia Test Interface

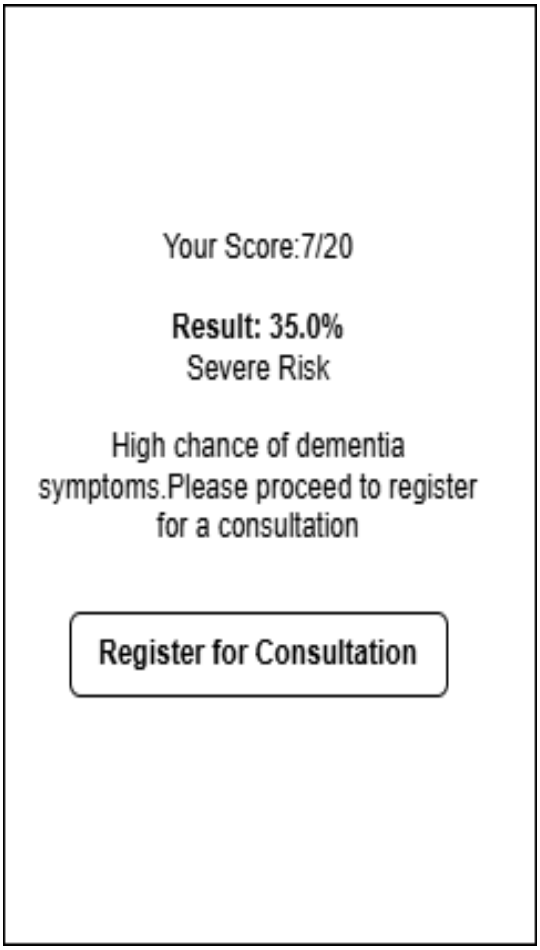
Task input page for ADHD patient



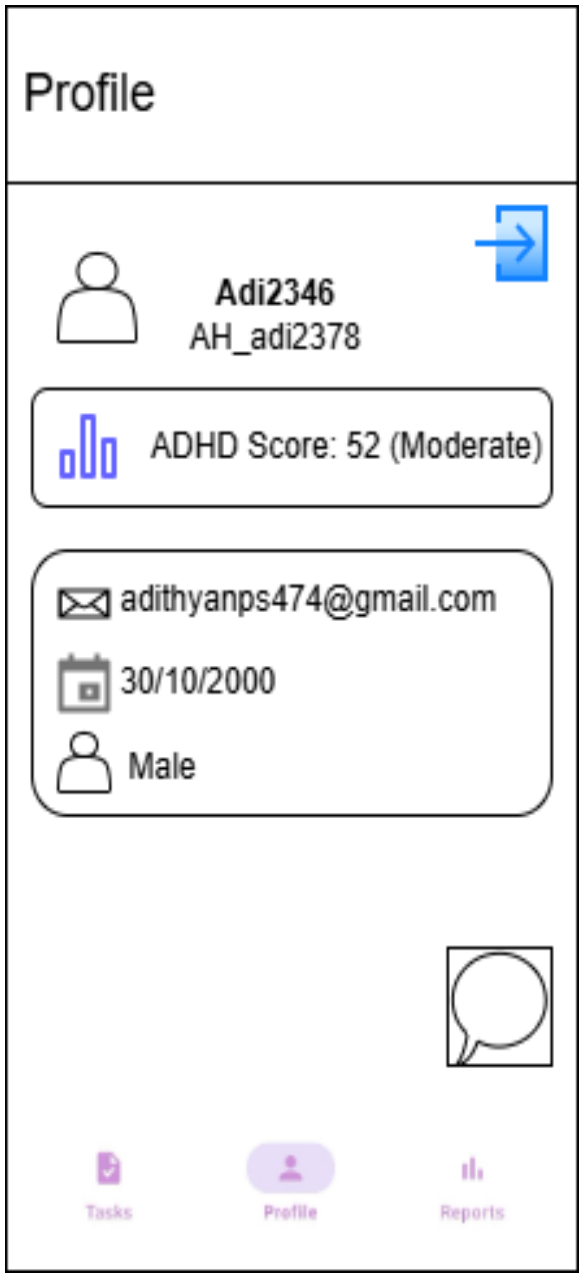
6.2 Output Design

Alarm interface

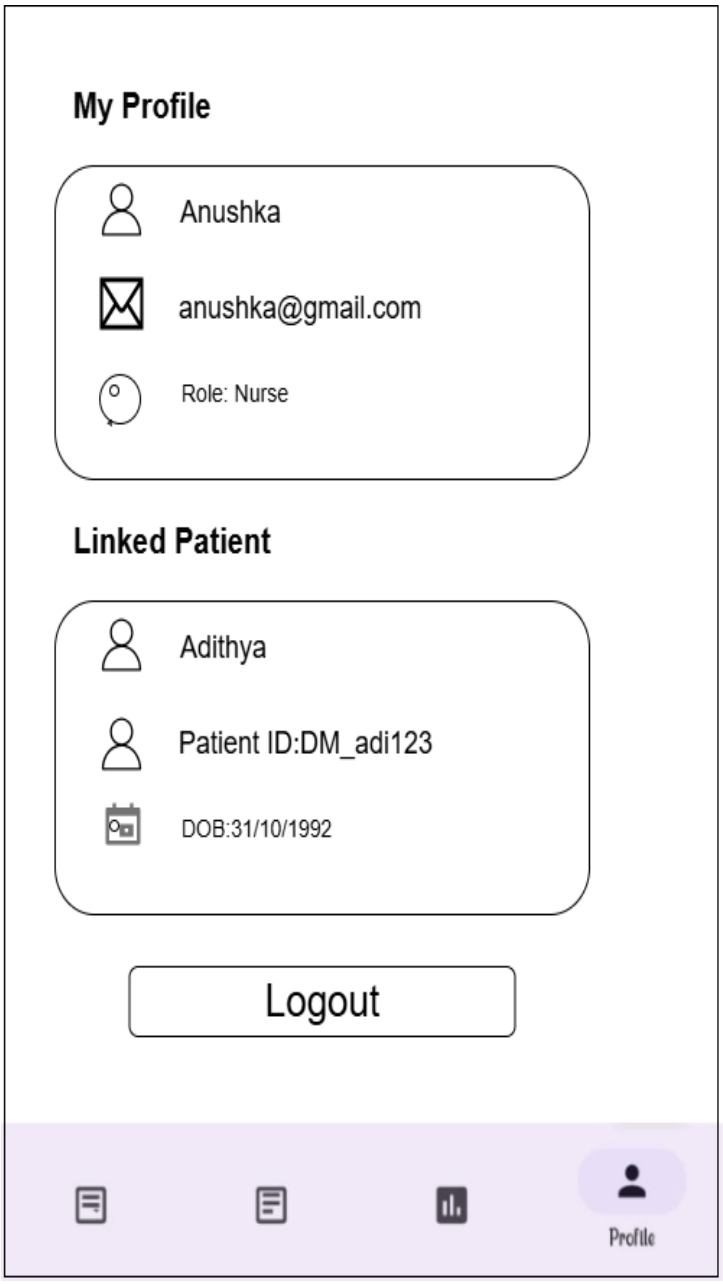
Dementia Result Page



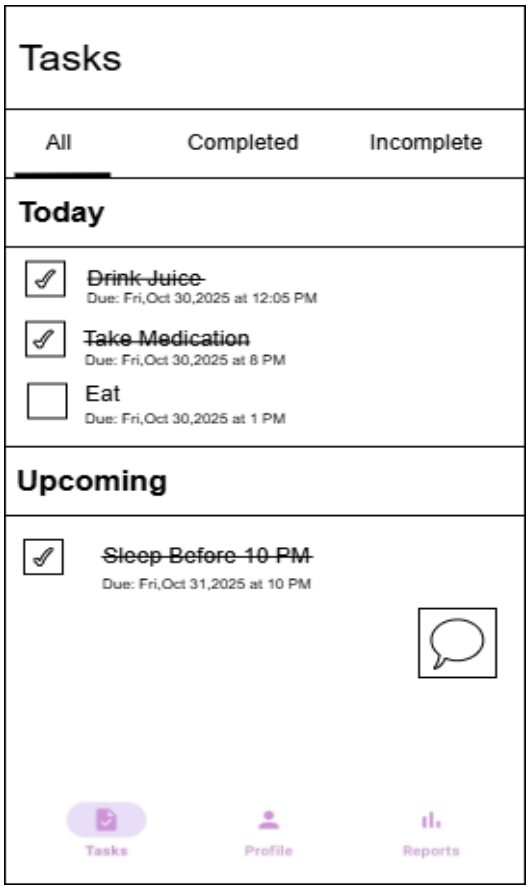
Patient Profile



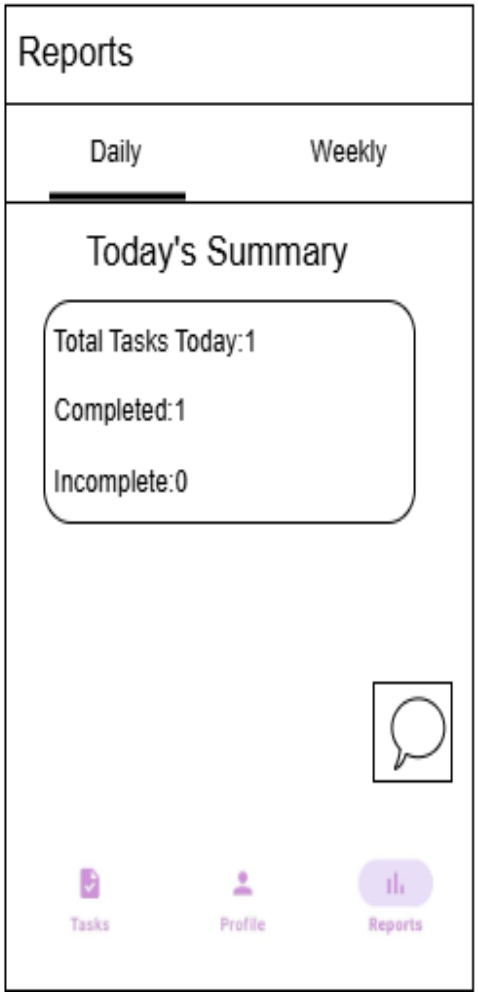
Caregiver Profile

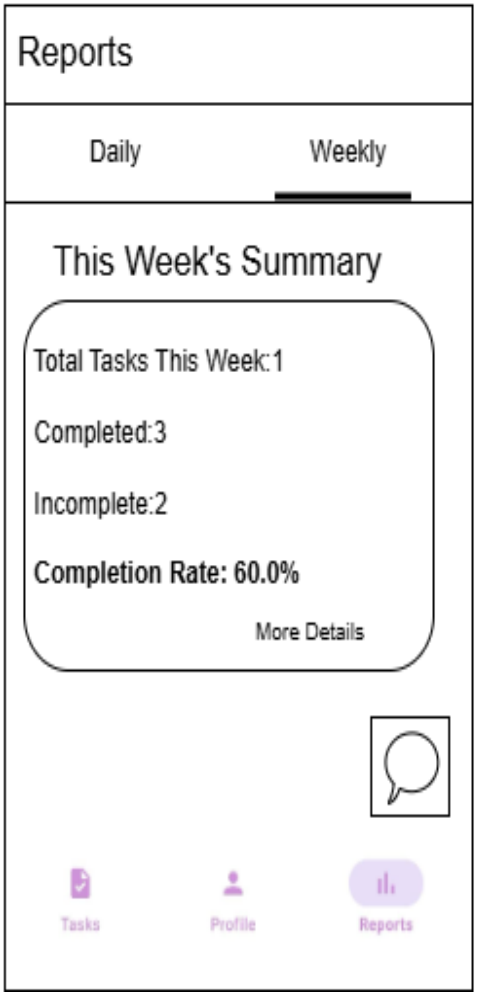


Dementia task interface

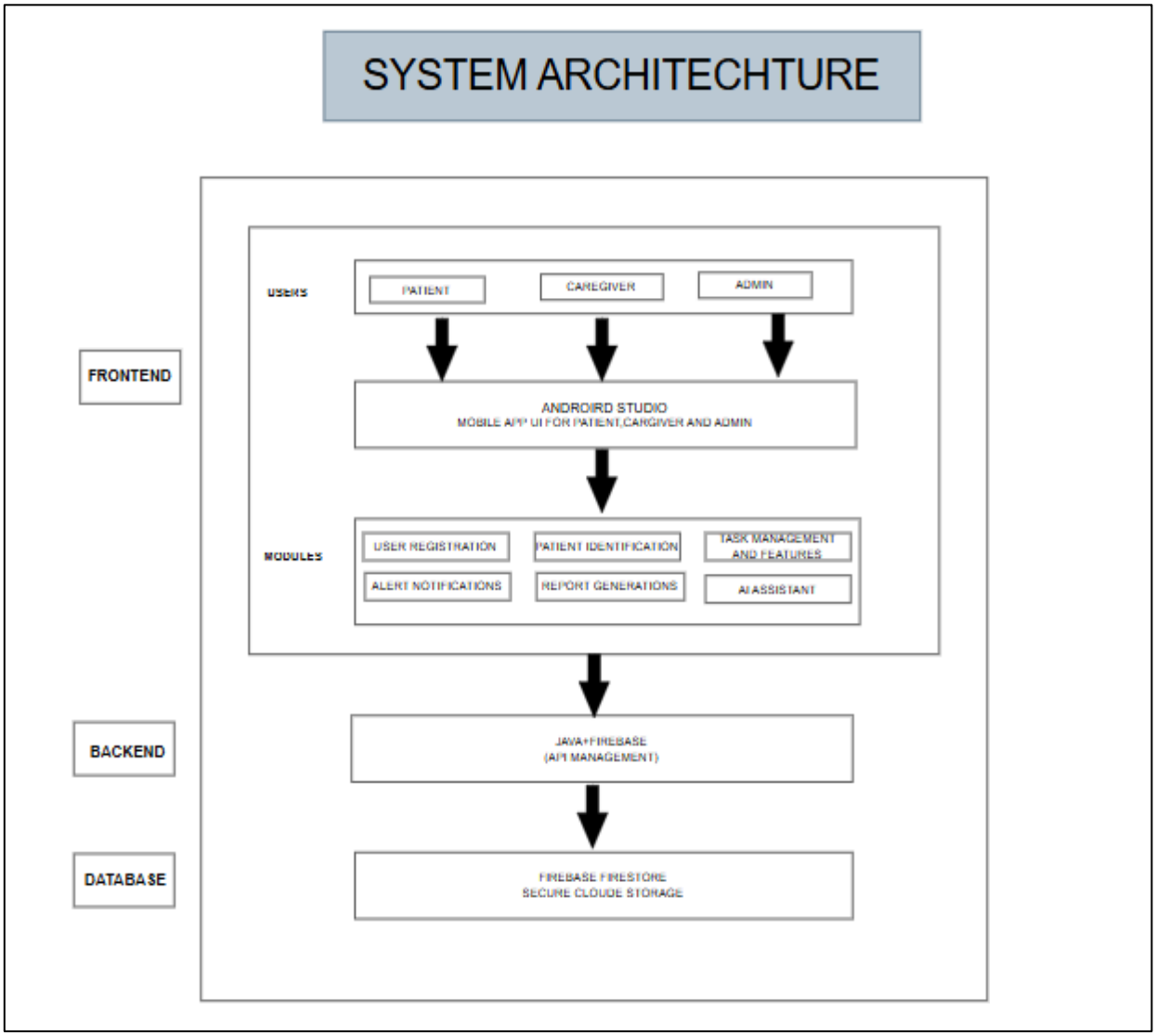


Daily and weekly reports



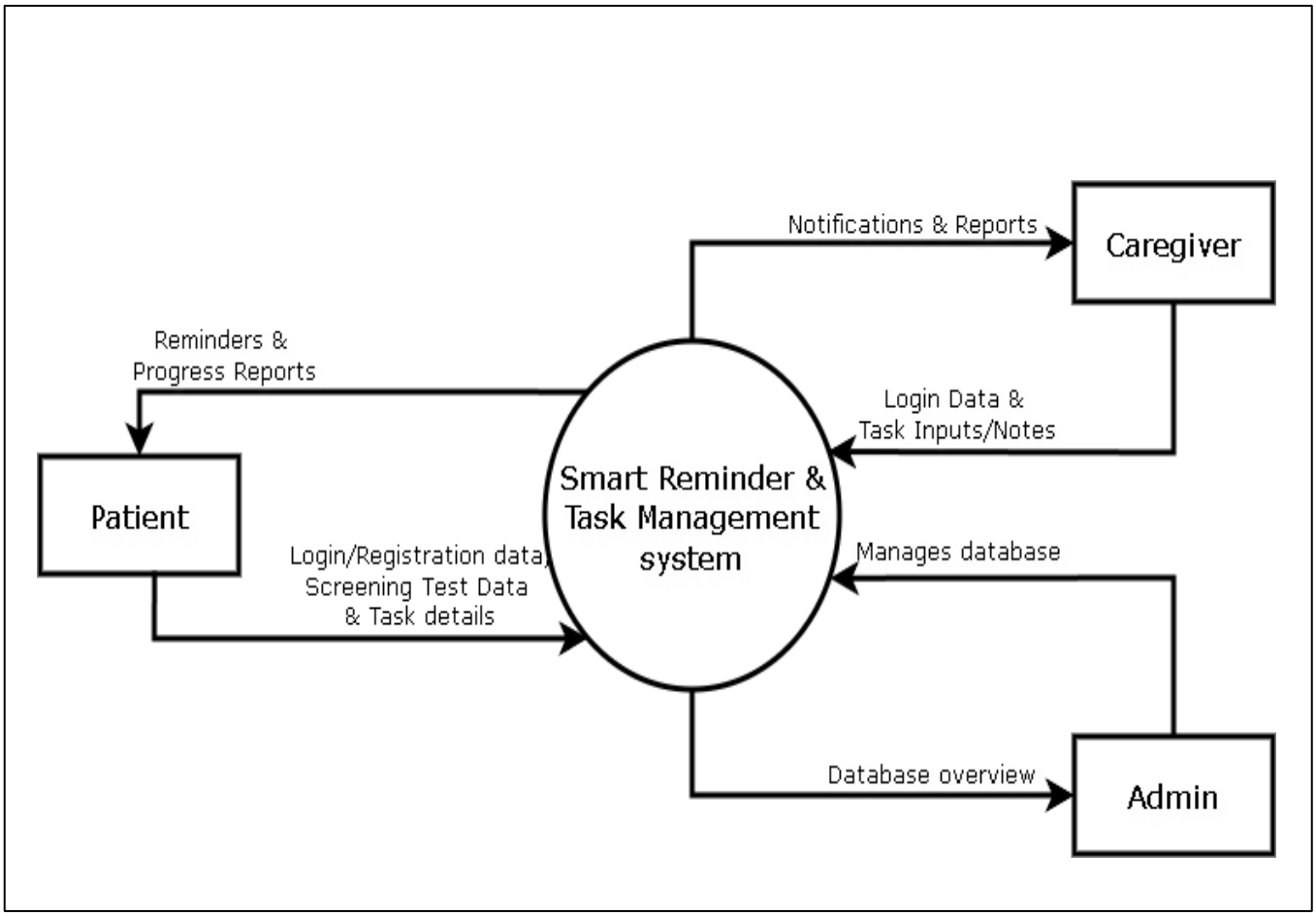


6.3 Architecture Design

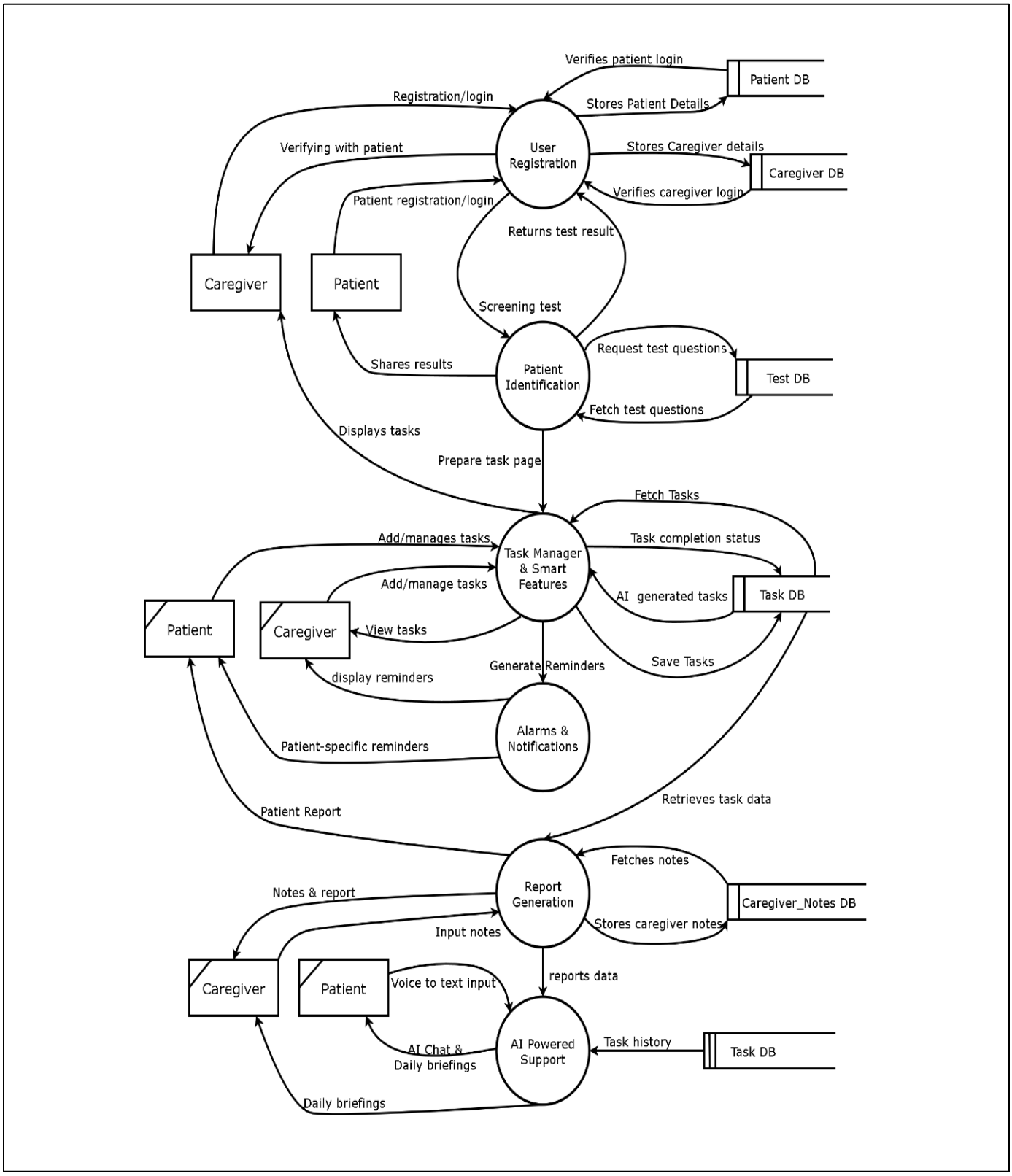


6.3.1 Data Flow Diagram

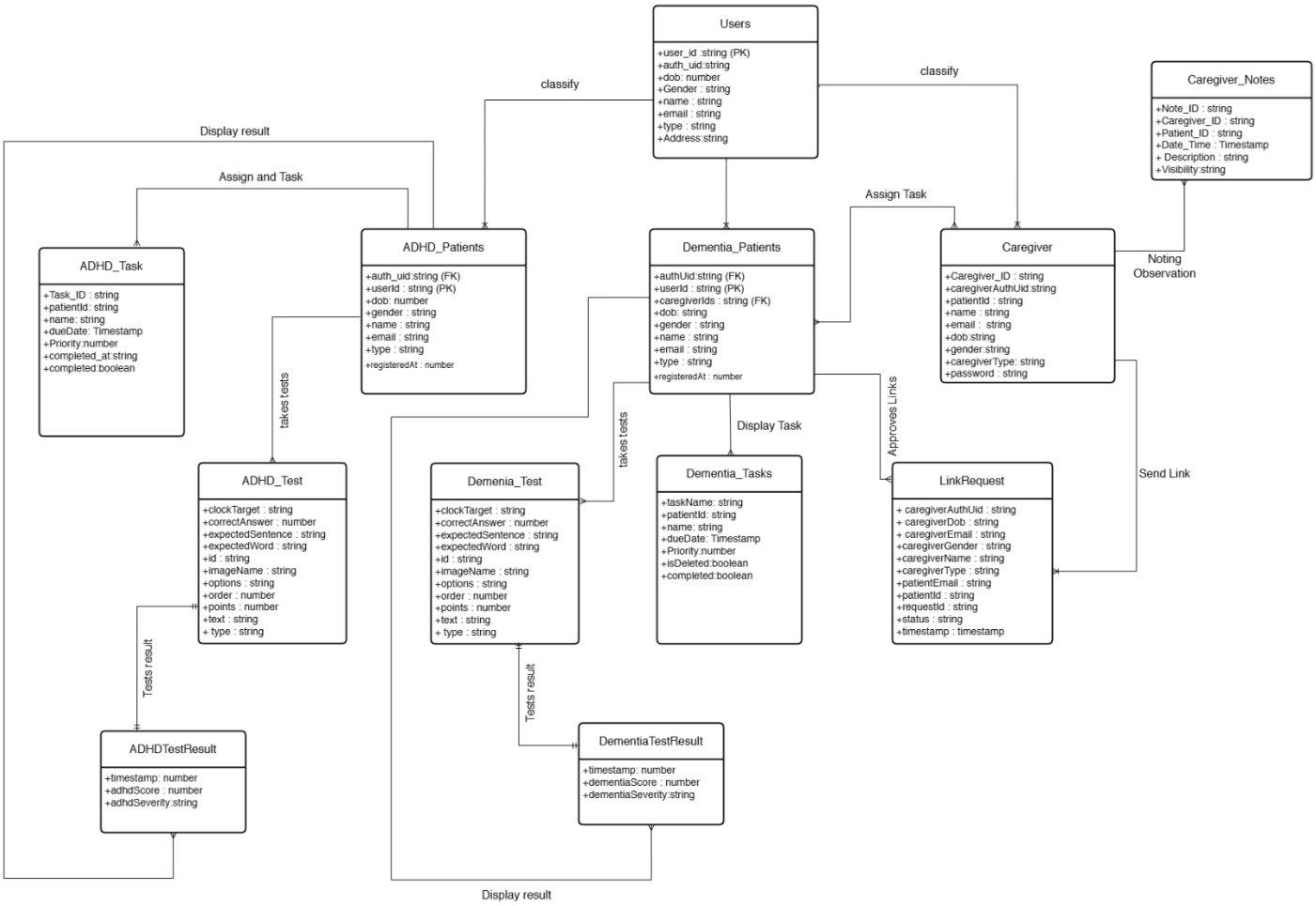
DFD Level 0



DFD Level 1


Verifies patient login
Patient DB
Registration/login
Stores Patient Details
User Registration
Verifying with patient
Stores Caregiver details
Caregiver DB
Verifies caregiver login
Patient registration/login
Returns test result
Caregiver
Patient
Screening test
Request test questions
Shares results
Patient Identification
Test DB
Fetch test questions
Displays tasks
Prepare task page
Fetch Tasks
Add/manages tasks
Task completion status
Task Manager & Smart Features
Add/manage tasks
AI generated tasks
Task DB
Patient
Caregiver
View tasks
Generate Reminders
Save Tasks
display reminders
Alarms & Notifications
Patient-specific reminders
Patient Report
Retrieves task data
Fetches notes
Report Generation
Notes & report
Caregiver_Notes DB
Input notes
Stores caregiver notes
Caregiver
Patient
Voice to text input
reports data
AI Chat & Daily briefings
AI Powered Support
Task history
Task DB
Daily briefings

6.3.2 ER Diagram



6.3.3 Table Design

Users

Attribute	Data Type	Key Type
user_id	VARCHAR	PRIMARY KEY, UNIQUE
auth_uid	VARCHAR	UNIQUE
name	VARCHAR	NOT NULL
email	VARCHAR	UNIQUE
dob	DATE	NULL
type	VARCHAR	NOT NULL

ADHD_Patients

Attribute	Data Type	Key Type
adhd_id	VARCHAR	PRIMARY KEY
auth_uid	VARCHAR	FOREIGN KEY
name	VARCHAR	NOT NULL
email	VARCHAR	UNIQUE
dob	DATE	NOT NULL
type	VARCHAR	NOT NULL

ADHD_Tasks

Attribute	Data Type	Key Type
task_id	VARCHAR	PRIMARY KEY
adhd_id	VARCHAR	FOREIGN KEY
task_date	DATE	NOT NULL
task_name	VARCHAR	NOT NULL
completed	BOOLEAN	DEFAULT FALSE

ADHD_Test_Results

Attribute	Data Type	Key Type
result_id	VARCHAR	PRIMARY KEY
adhd_id	VARCHAR	FOREIGN KEY
adhdScore	INT	NOT NULL
adhdSeverity	VARCHAR	NOT NULL

Dementia_Patients

Attribute	Data Type	Key Type
dementia_id	VARCHAR	PRIMARY KEY
auth_uid	VARCHAR	FOREIGN KEY
name	VARCHAR	NOT NULL
email	VARCHAR	UNIQUE
caregiver_ids	TEXT	NULL
type	VARCHAR	NOT NULL

Dementia_Tasks

Attribute	Data Type	Key Type
task_id	VARCHAR	PRIMARY KEY
dementia_id	VARCHAR	FOREIGN KEY
task_date	DATE	NOT NULL
task_name	VARCHAR	NOT NULL
description	TEXT	NULL
completed	BOOLEAN	DEFAULT FALSE

Dementia_Test_Results

Attribute	Data Type	Key Type
result_id	VARCHAR	PRIMARY KEY
dementia_id	VARCHAR	FOREIGN KEY
dementiaScore	INT	NOT NULL
dementiaSeverity	VARCHAR	NOT NULL

Caregivers

Attribute	Data Type	Key Type
caregiver_id	VARCHAR	PRIMARY KEY
caregiver_auth_uid	VARCHAR	FOREIGN KEY
name	VARCHAR	NOT NULL
email	VARCHAR	UNIQUE
caregiver_type	VARCHAR	NOT NULL
patient_id	VARCHAR	FOREIGN KEY

CaregiverNotes

Attribute	Data Type	Key Type
note_id	VARCHAR	PRIMARY KEY
caregiver_id	VARCHAR	FOREIGN KEY
client_id	VARCHAR	FOREIGN KEY
content	TEXT	NULL

LinkRequests

Attribute	Data Type	Key Type
request_id	VARCHAR	PRIMARY KEY
caregiver_id	VARCHAR	FOREIGN KEY
patient_id	VARCHAR	FOREIGN KEY
status	VARCHAR	NOT NULL

Tests

Attribute	Data Type	Key Type
test_id	VARCHAR	PRIMARY KEY
question_id	VARCHAR	UNIQUE
question_text	TEXT	NOT NULL
options	TEXT	NULL

7.0 System Testing and Implementation

7.1 System Testing

System testing is the process of evaluating the complete and integrated software to ensure that it meets the specified requirements and functions as intended. In the **Memora** project, testing was conducted continuously during and after each development stage, following the incremental development approach. Each module was tested individually and later integrated into the overall system.

Types of Testing Performed:

Unit Testing:
Each module, such as registration, task management, chatbot, and report generation, was tested separately to verify that the functions and logic worked correctly without dependency conflicts.

Integration Testing:
After individual testing, the modules were combined and tested as a group to ensure seamless interaction between components—for example, verifying that tasks created in the task module were correctly reflected in reports and synchronized with caregiver dashboards.

Functional Testing:
Focused on validating all user functions such as adding, updating, or deleting tasks, generating reports, and verifying reminder notifications and alarms.

User Interface Testing:
Checked for UI consistency, layout clarity, and ease of use, ensuring that the app remained accessible to users with cognitive impairments.

Performance Testing:
The application was tested on various Android devices to ensure smooth operation, low memory usage, and quick response times. Firebase synchronization and notification delivery speeds were also tested under different network conditions.

Security Testing:
Ensured data safety and secure access to user information through Firebase authentication. Tests confirmed that user data is securely stored and only accessible to authorized users.

Testing Results:

All modules were successfully validated after multiple iterations. Minor UI glitches and notification timing issues identified during early testing were resolved in later builds. The system demonstrated stable performance, accuracy in task handling, and reliable real-time synchronization with Firebase.

7.2 Maintenance

System maintenance is the final and continuous phase of the software development life cycle, ensuring that the deployed system remains functional, efficient, and adaptable to user needs over time. For Memora, maintenance plays a crucial role in improving reliability, resolving post-deployment issues, and enhancing user experience based on real-world feedback.

Types of Maintenance Applied:

1. **Corrective Maintenance:**
   This involves identifying and fixing bugs or issues that appear during real-world usage. Examples include correcting timing delays in notifications, improving data synchronization with Firebase, and resolving UI misalignments across devices.
2. **Adaptive Maintenance:**
   As technologies and APIs evolve, the system needs to stay compatible. Memora's adaptive maintenance includes updating APIs (such as Firebase SDKs or the KPI voice-to-text API), ensuring compatibility with new Android versions, and optimizing the app for newer device resolutions.
3. **Perfective Maintenance:**
   Based on user feedback and performance evaluation, continuous improvements are made to enhance usability and performance. This includes refining the chatbot's response accuracy, improving task classification logic, and simplifying the caregiver interface.
4. **Preventive Maintenance:**
   Routine code reviews and optimization are conducted to prevent potential failures or performance degradation. This ensures database efficiency, stable cloud connections, and consistent alarm scheduling.

Maintenance Approach:

- Regular performance checks are conducted using Android Studio's profiler tools.
- Backup procedures ensure that all data stored in Firebase remains safe and recoverable.
- Feedback from users and caregivers is periodically collected to identify new features or issues.
- Updated APKs are deployed after each version review to maintain functionality and reliability.

Outcome:

Through consistent maintenance, Memora continues to deliver a smooth, secure, and reliable experience for ADHD and dementia users. The team's proactive approach ensures that the app adapts to user needs, technological changes, and long-term usability requirements.

8.0 Conclusion

Memora is more than just a task management tool; it is a dedicated cognitive support system tailored to the unique needs of ADHD and early-stage dementia users. By merging accessibility, AI-powered personalization, and caregiver collaboration into a single, free-to-use platform, it stands apart from costly and restrictive alternatives.

This system aims to restore independence, confidence, and structure to user's daily lives, while also reducing caregiver burden. With future expansion into additional cognitive support features, integration with emerging technologies, and a commitment to accessibility for all income levels, Memora has the potential to make a lasting impact on the way cognitive care is delivered. It represents not just a technological achievement, but a vision for inclusive, compassionate, and effective digital healthcare.

9.0 Scope for Further Development

The system has strong potential for future expansion:

- Wearable Device Integration: Sync with smartwatches and fitness bands for discreet reminders and basic health tracking.
- Gamification: Rewards and achievement systems to improve engagement for ADHD users.
- Data Analytics: Trend analysis for healthcare professionals.
- Accessibility Enhancements: Gesture controls and multilingual support.
- Support for Additional Cognitive Conditions: Expansion to include features for conditions like colour blindness, mild anxiety, or other cognitive disorders.
- Sensory-Friendly Mode: Adjustable visuals and animations for sensitive users.
- Offline Support: Reminders and task logging work without an internet connection.

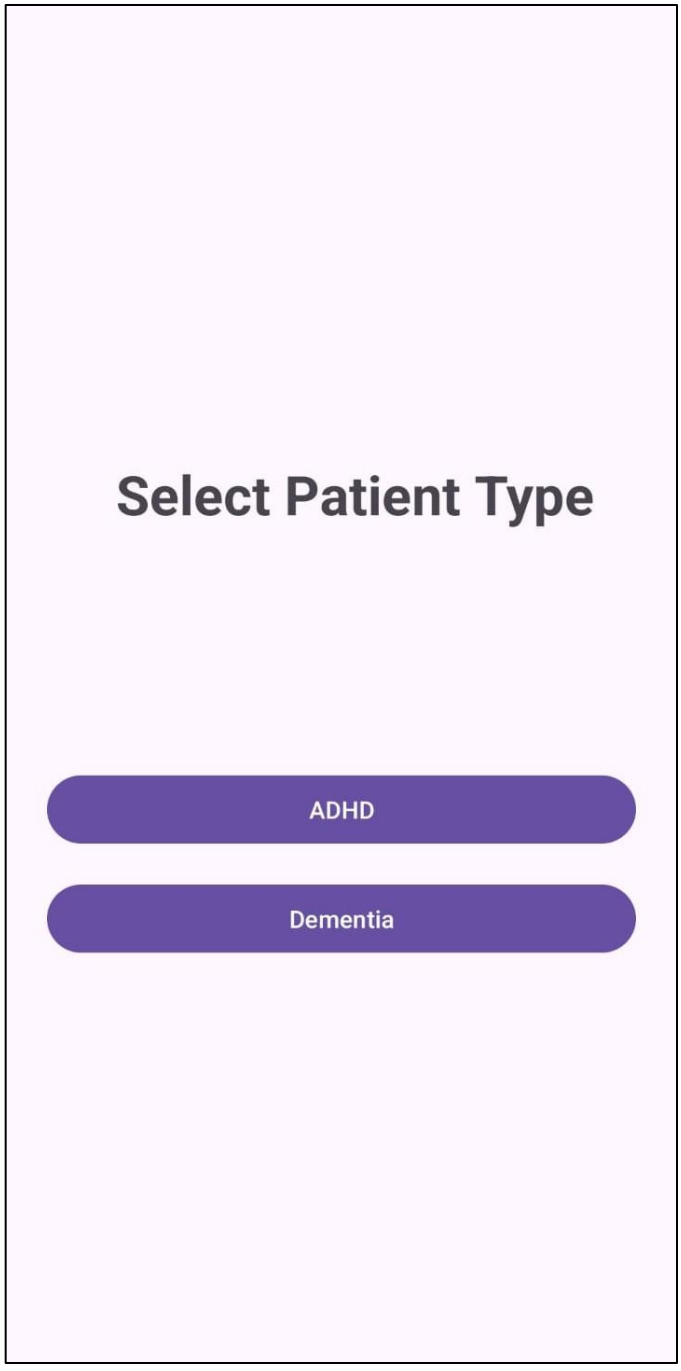
10.0 Bibliography

1. World Health Organization (WHO). Dementia Fact Sheet, 2021.
2. American Psychiatric Association. Diagnostic and Statistical Manual of Mental Disorders (DSM-5), 2013.
3. MoCA Test: Montreal Cognitive Assessment, https://www.mocatest.org
4. Adult ADHD Self-Report Scale (ASRS) v1.1: WHO Collaborative International Study.
5. Firebase Documentation: https://firebase.google.com/docs
6. Android Developer Documentation: https://developer.android.com
7. VOSK Speech Recognition API: Alpha Cephei: https://alphacephei.com/vosk
8. Google Material Design Guidelines: https://material.io/design
9. Stack Overflow community discussions and code references.
10. TickTick App and Finch App user feedback (Play Store reviews, 2024–2025).

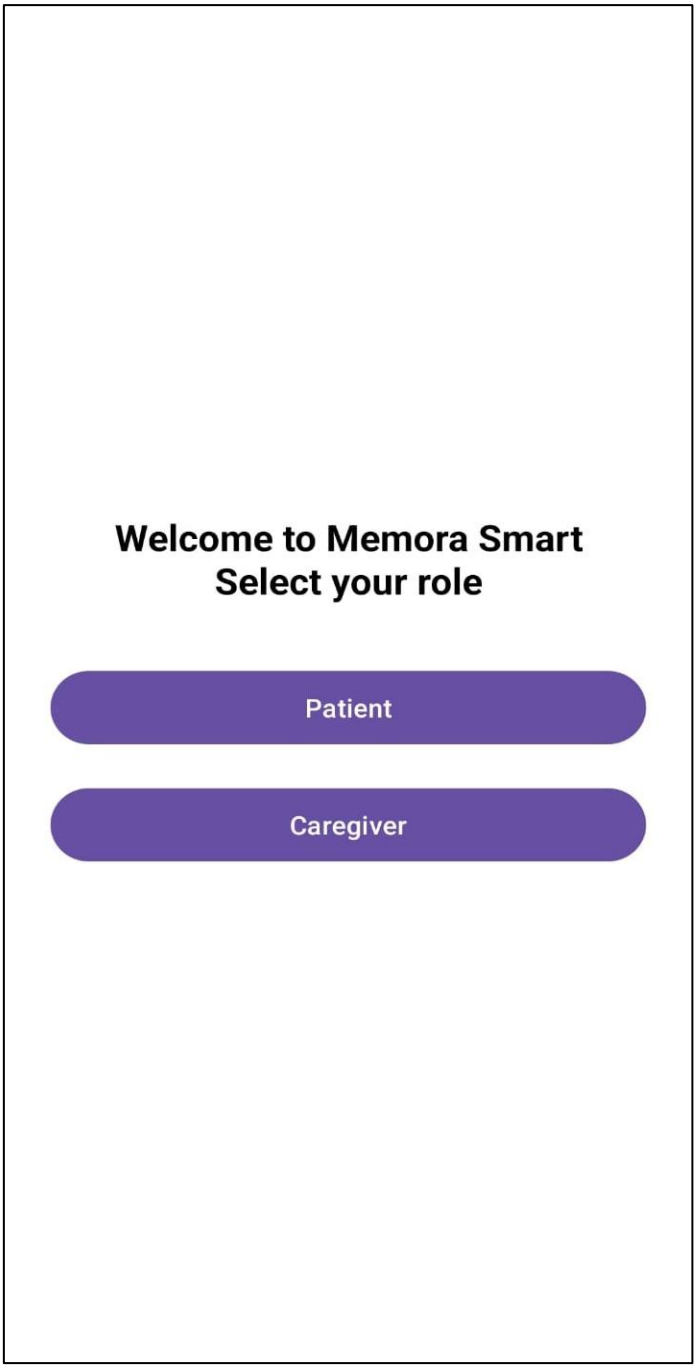
11.0 Appendix

11.1 Screen Shots

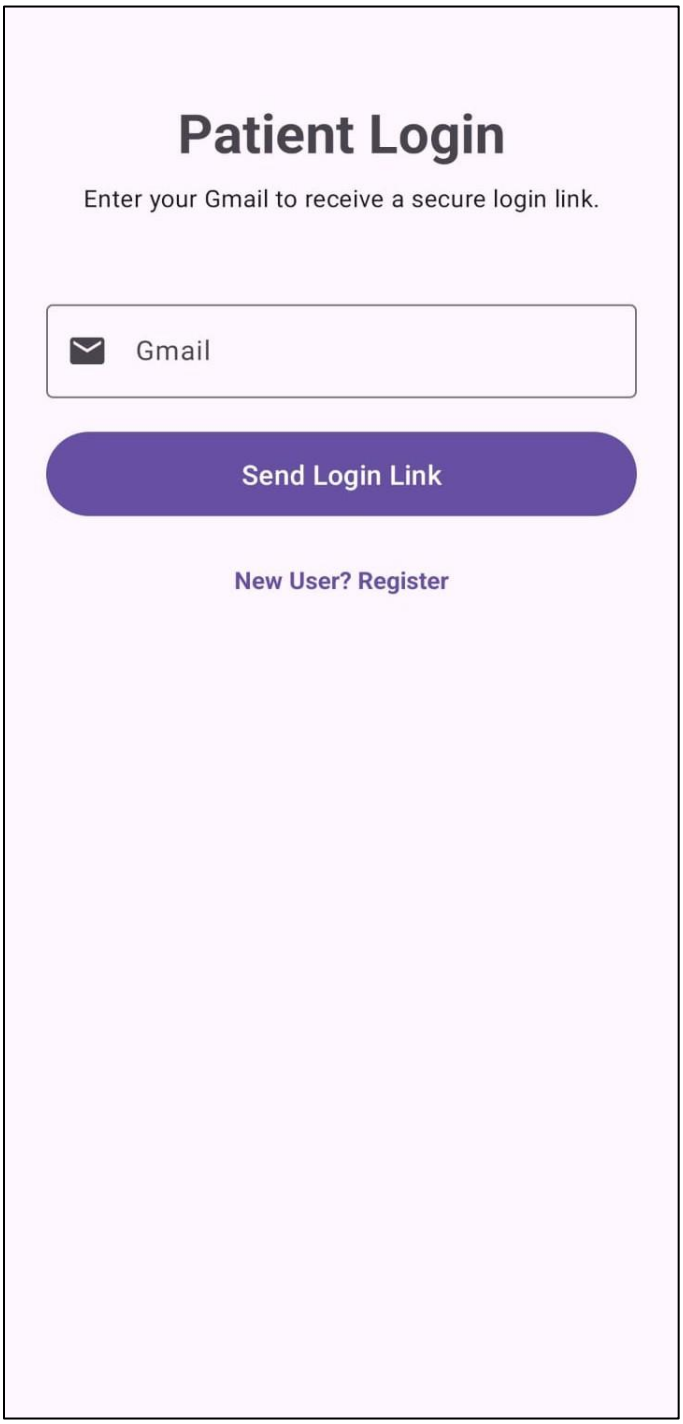
Patient selection



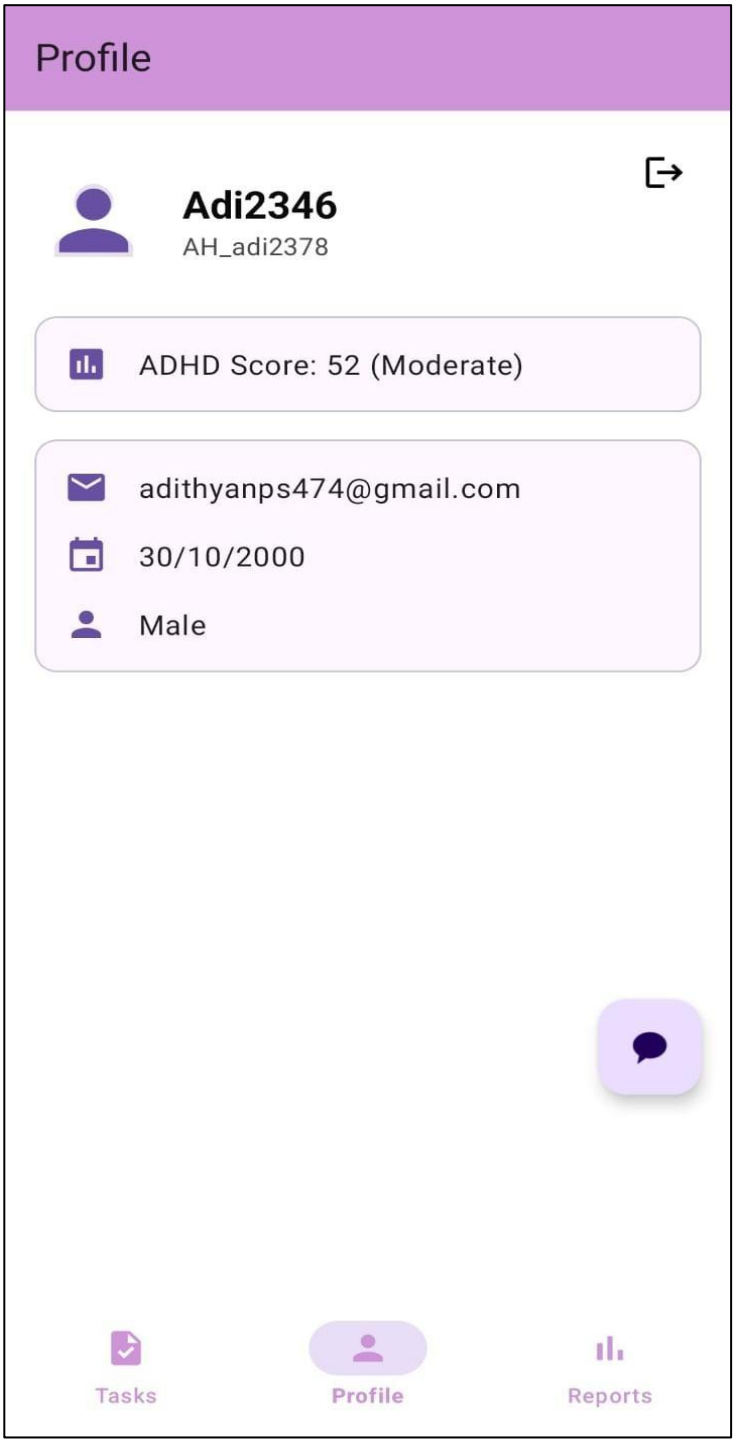
User Role selection



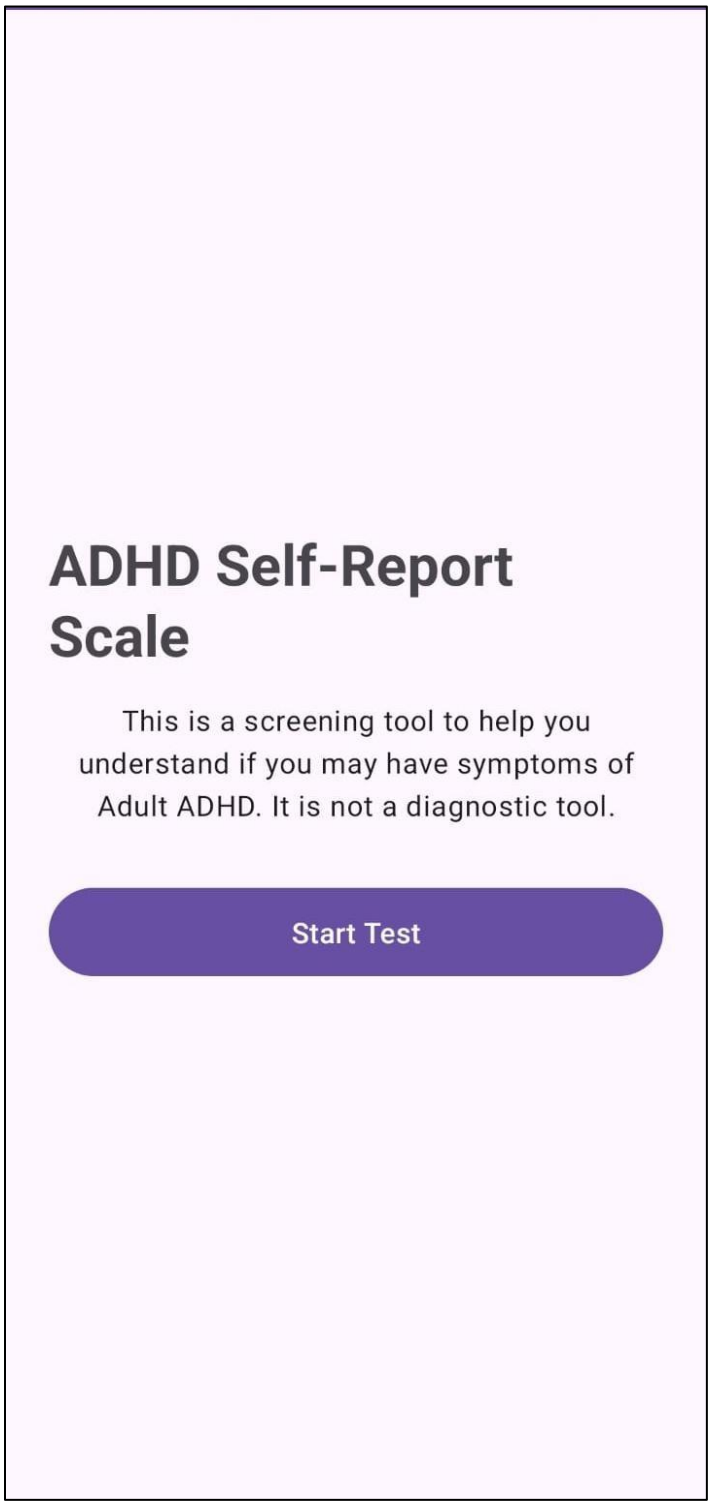
Patient Login



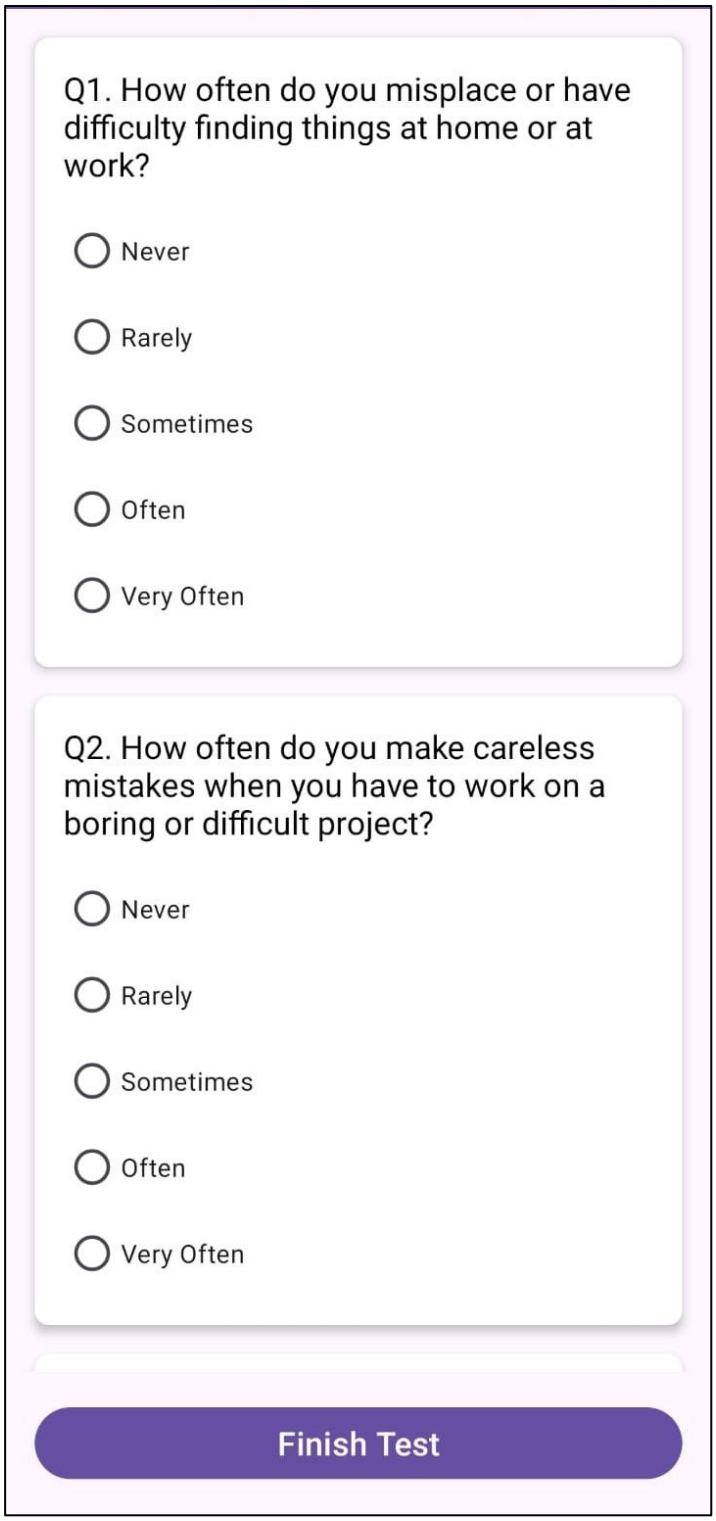
Patient Profile



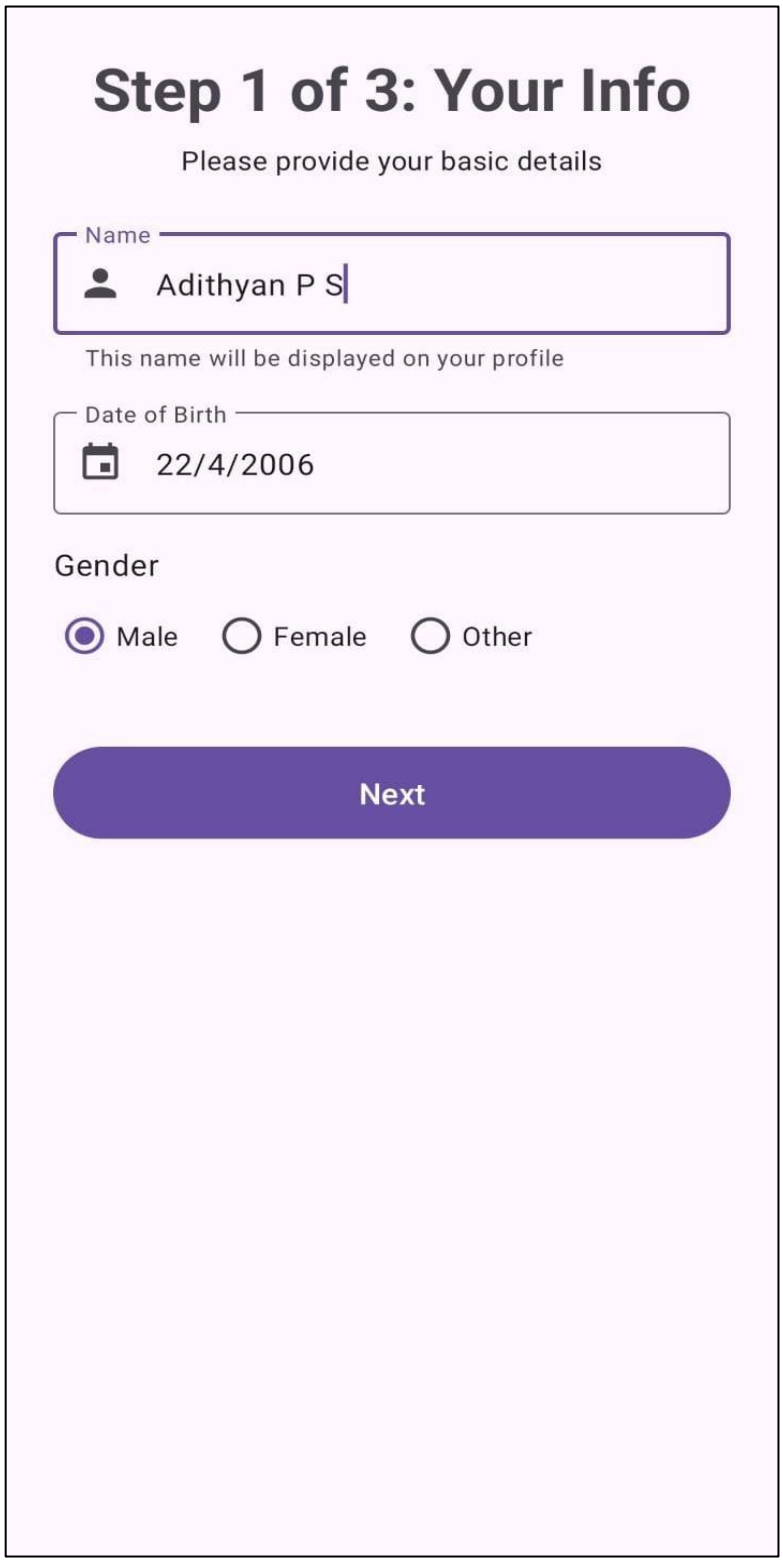
Test disclaimer



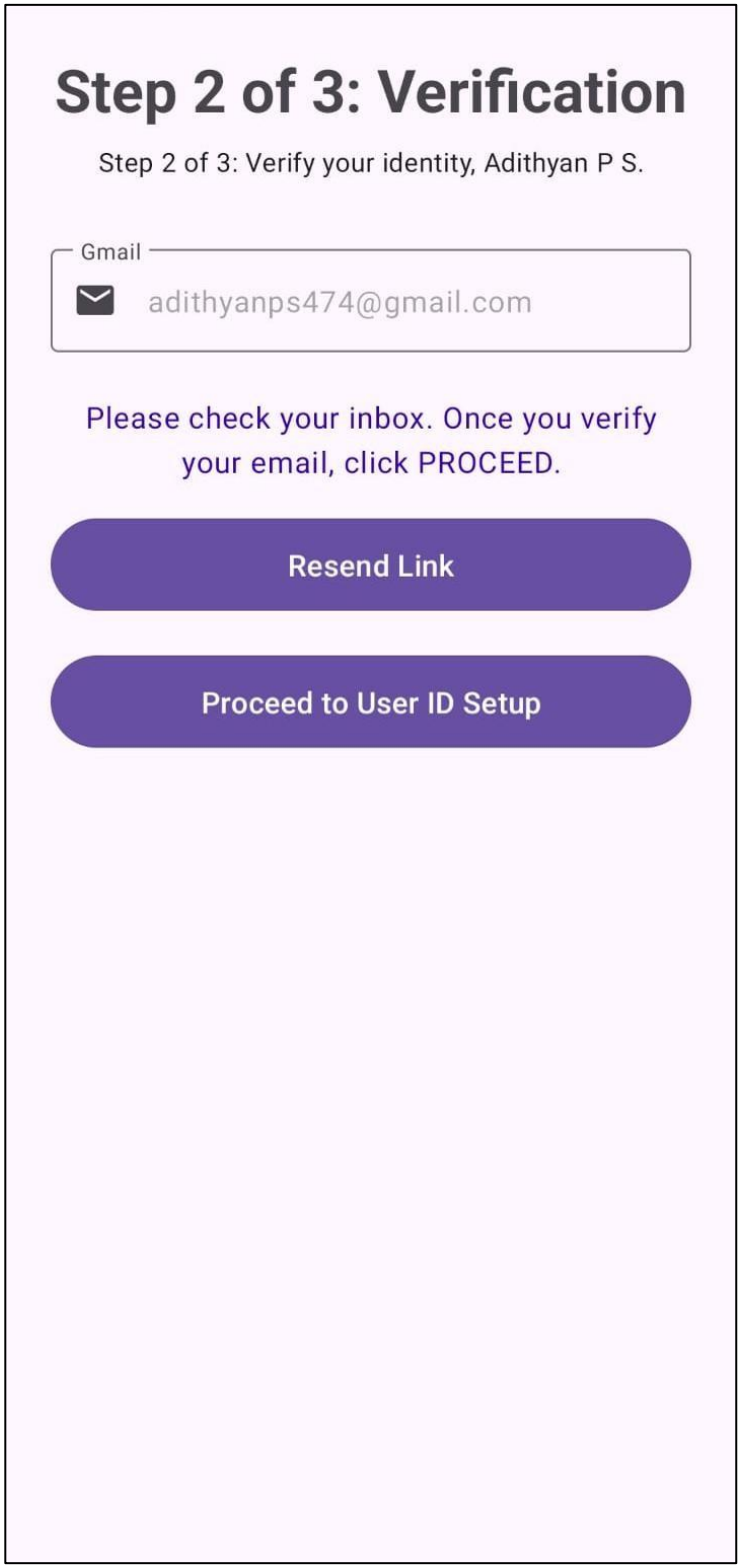
ADHD Test



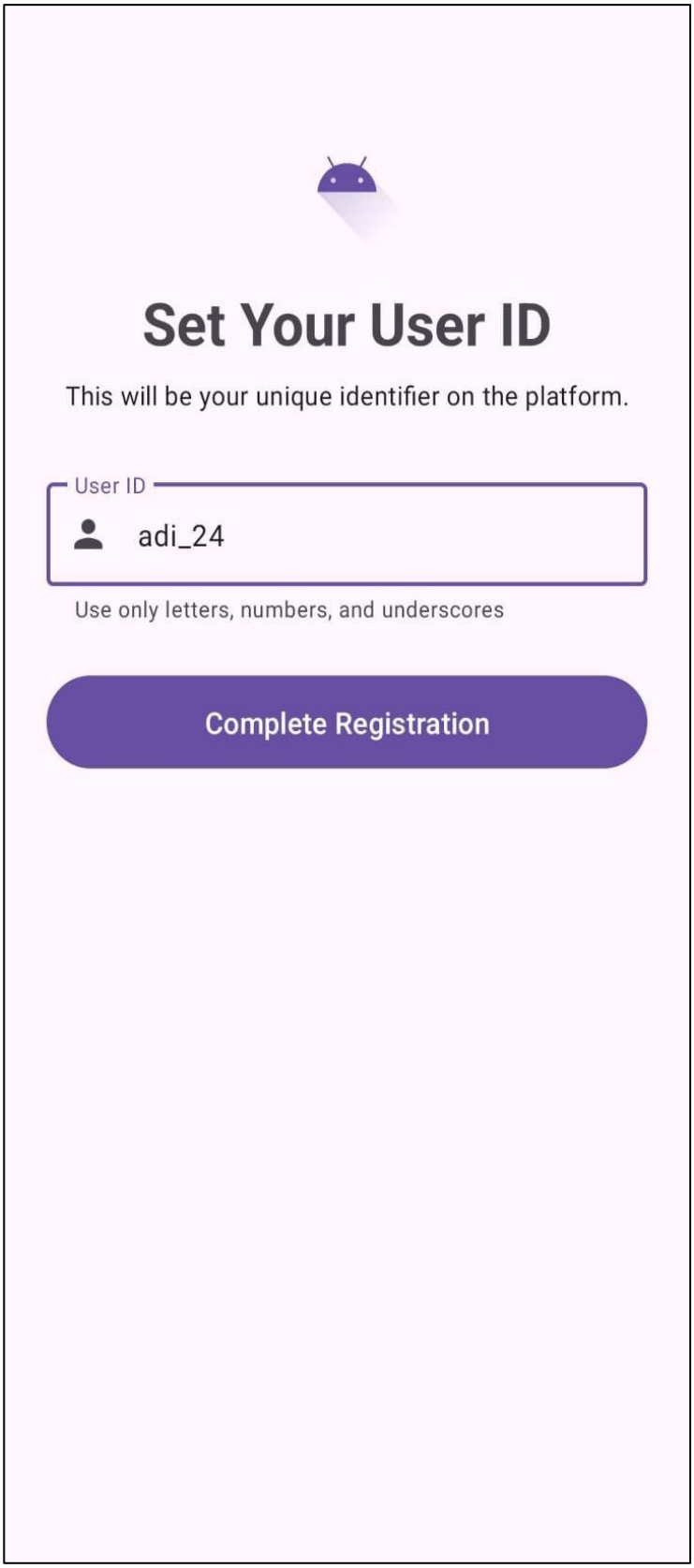
Registration step 1



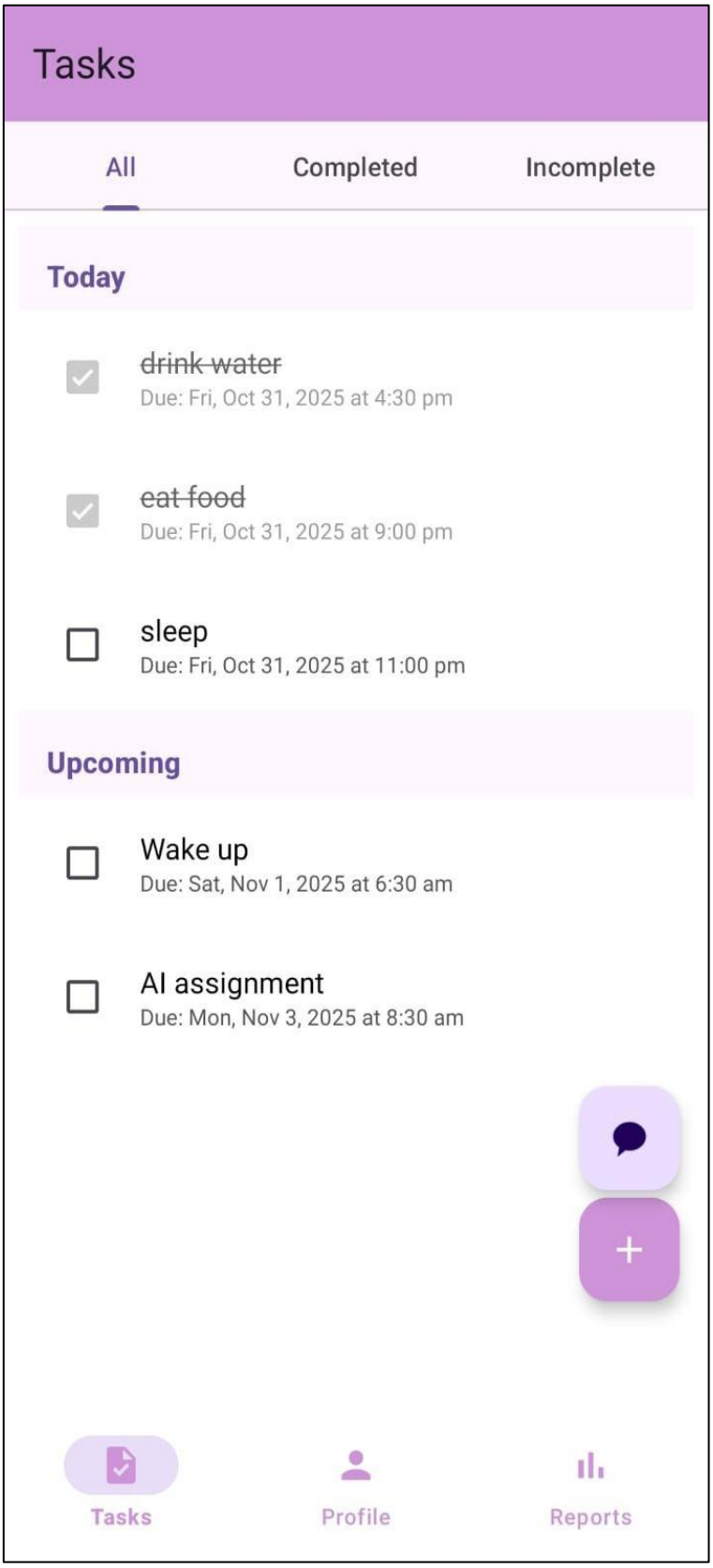
Registration step 2



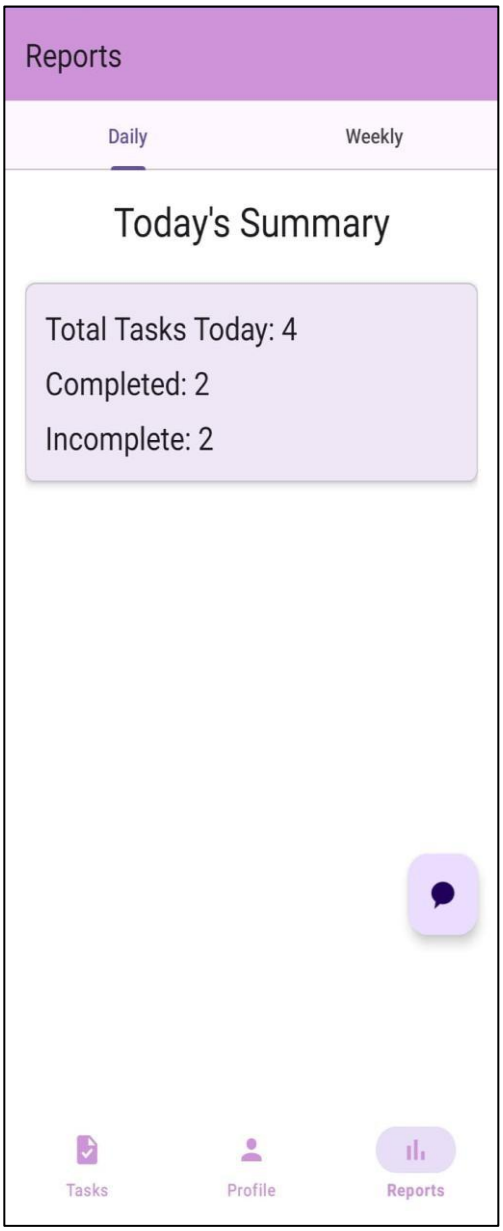
Registration step 3

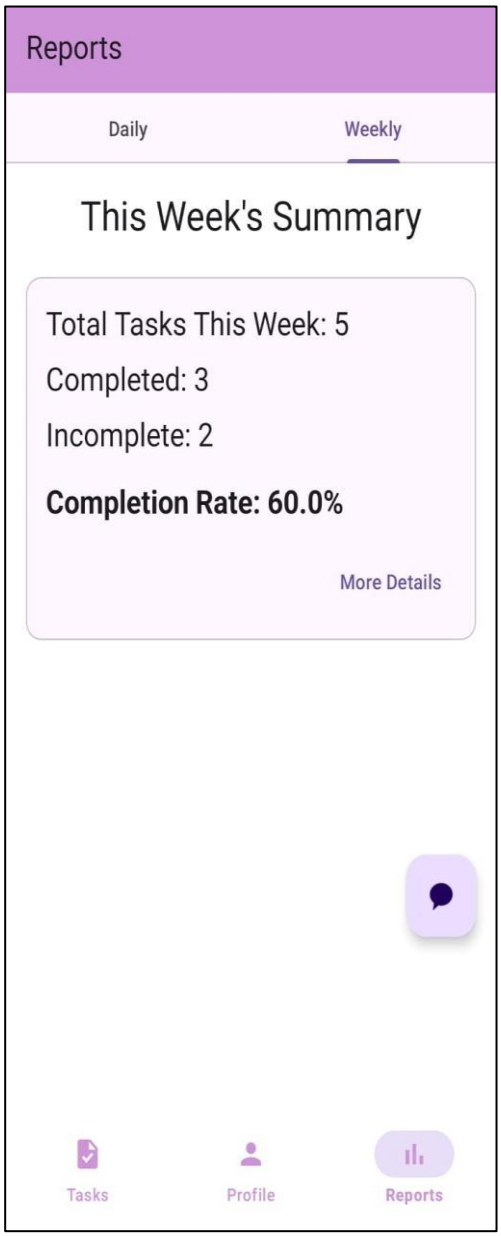


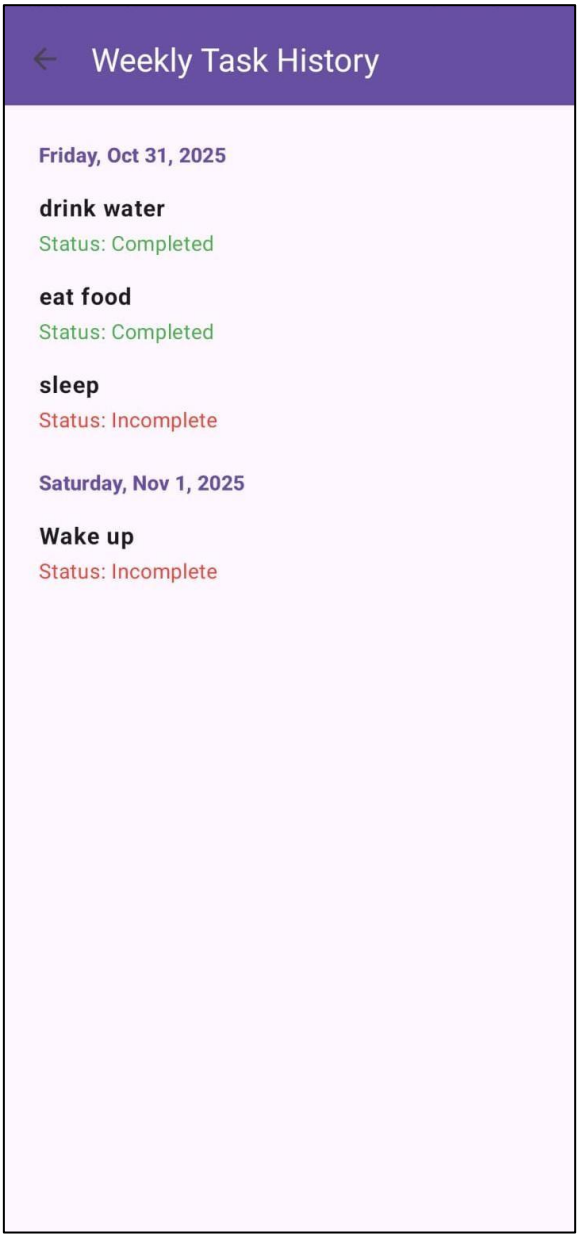
Patient task view



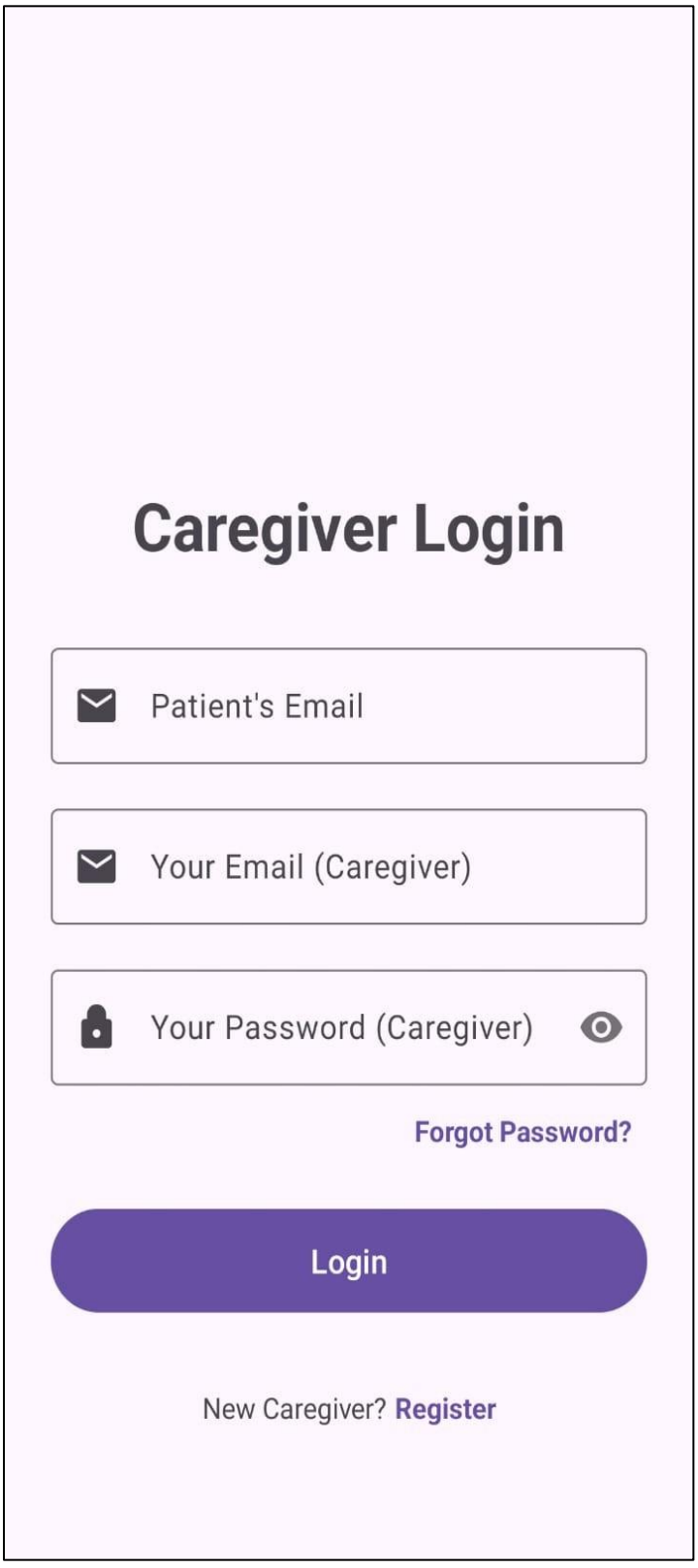
Daily and weekly reports along with weekly history



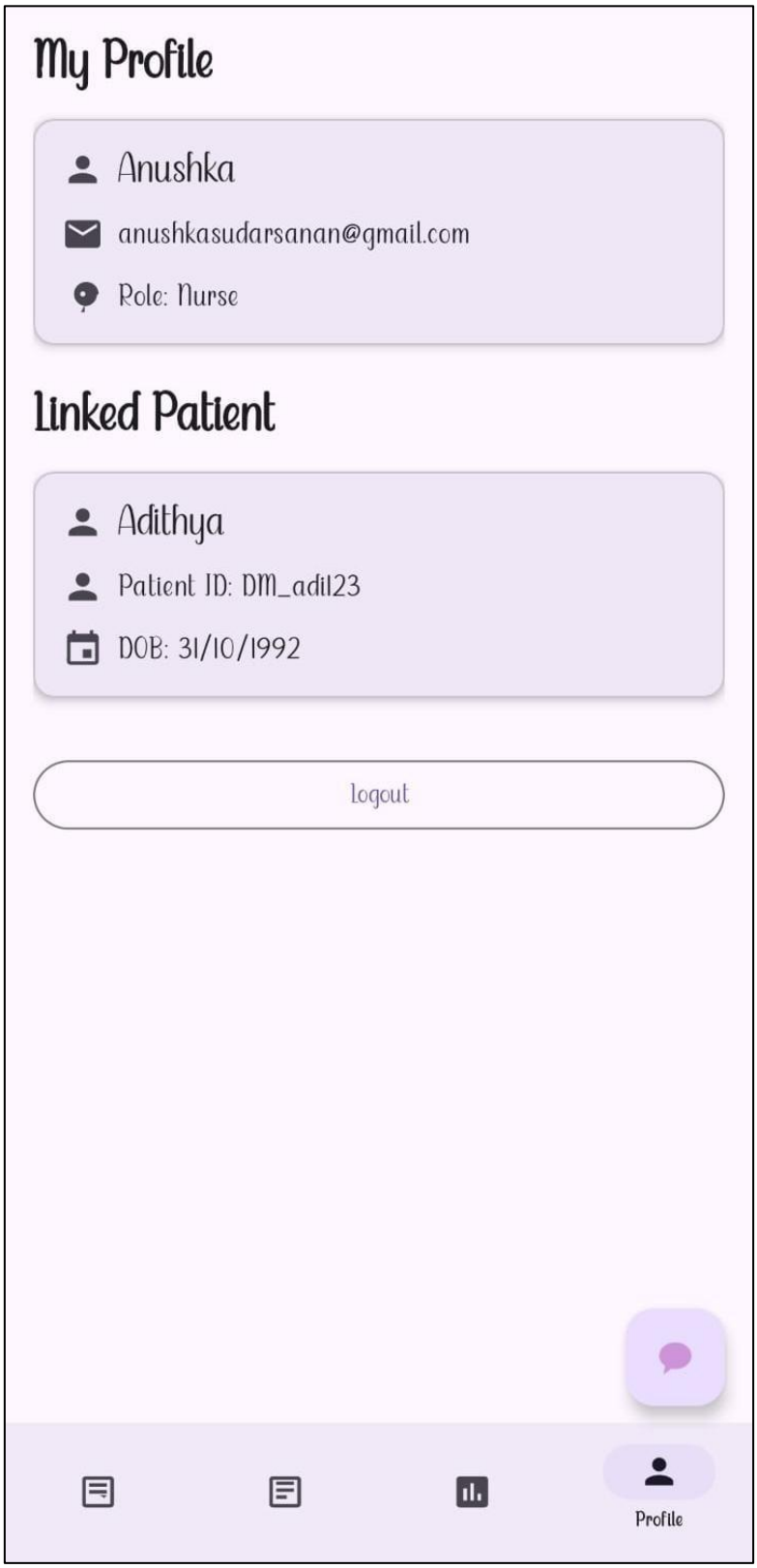




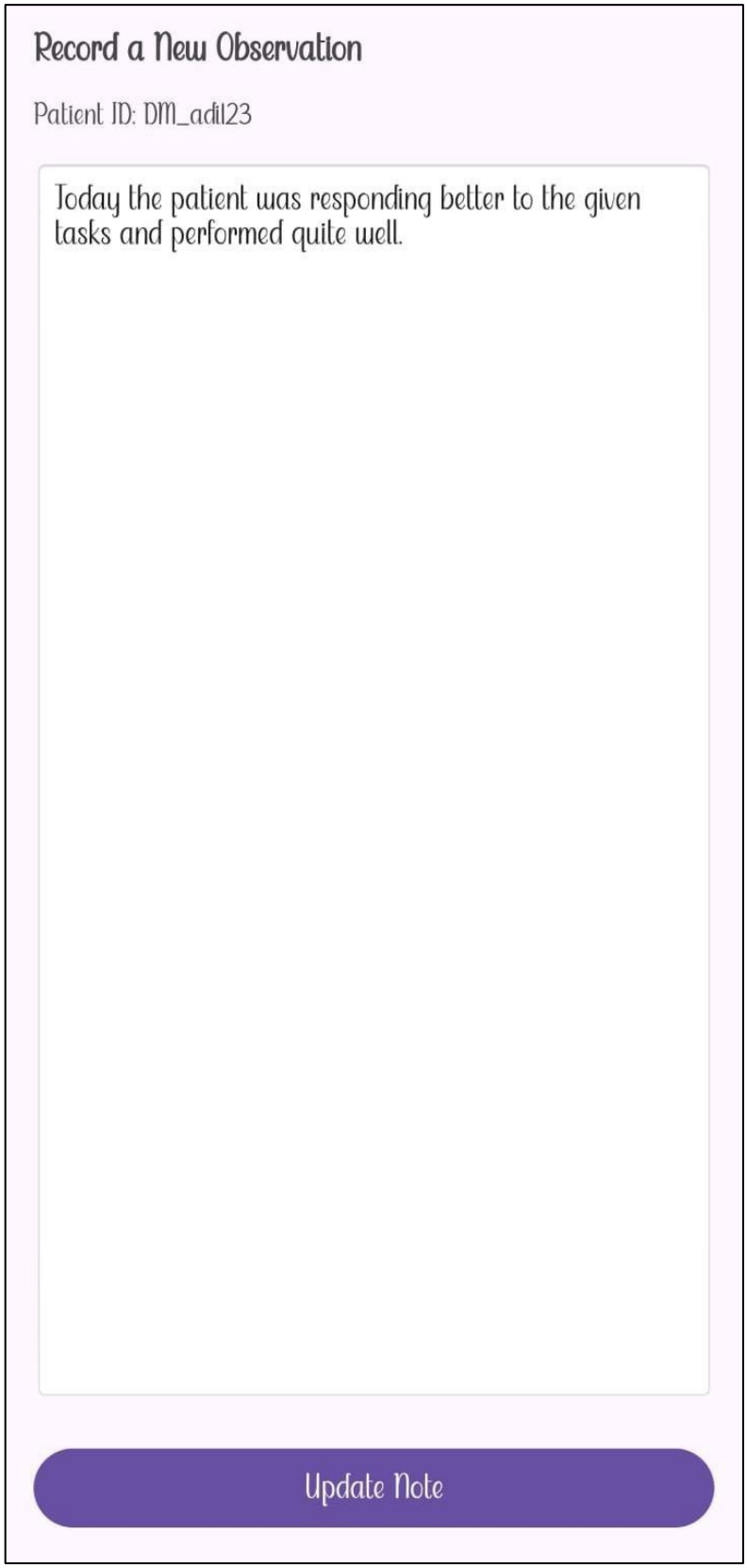
Caregiver login



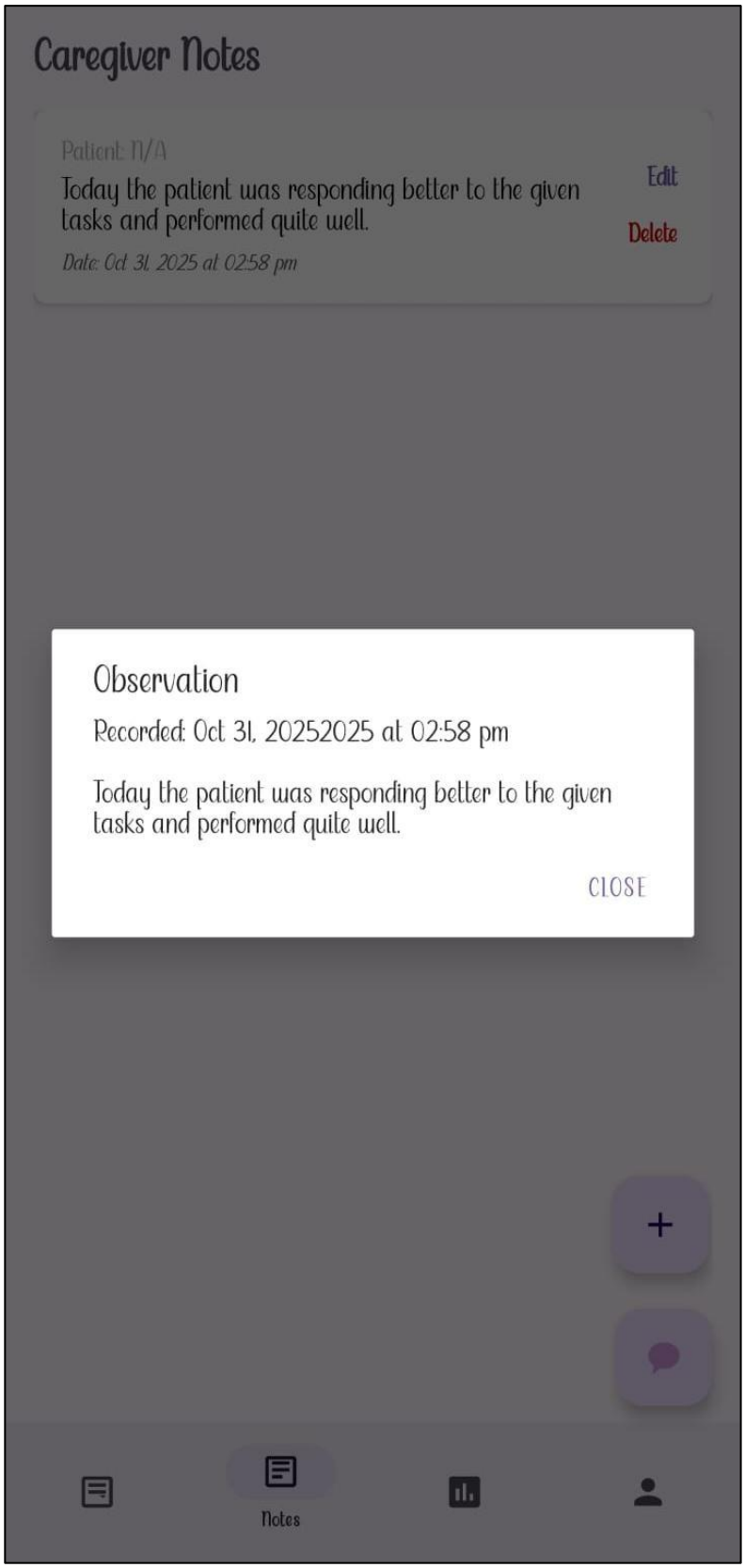
Caregiver Profile



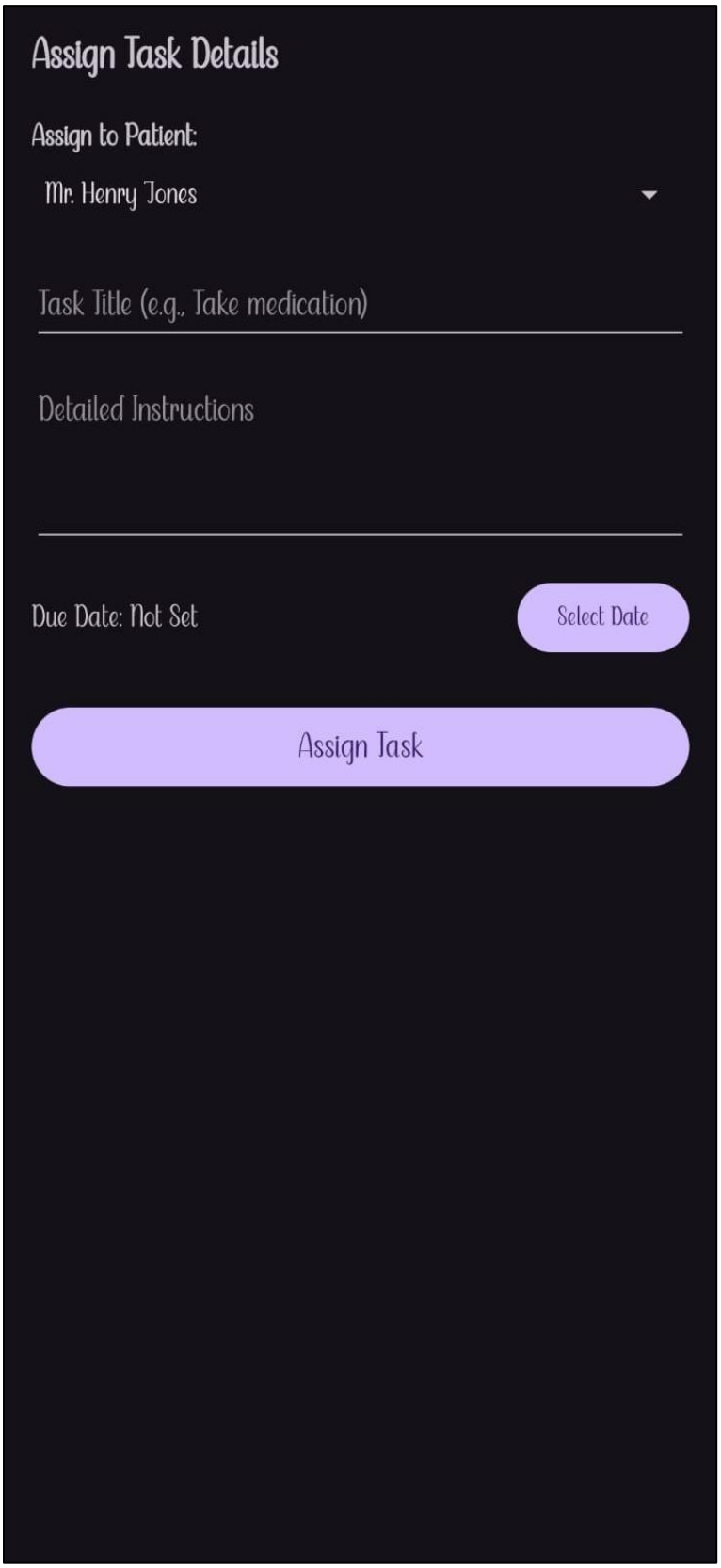
Updating caregiver notes



Notes page view

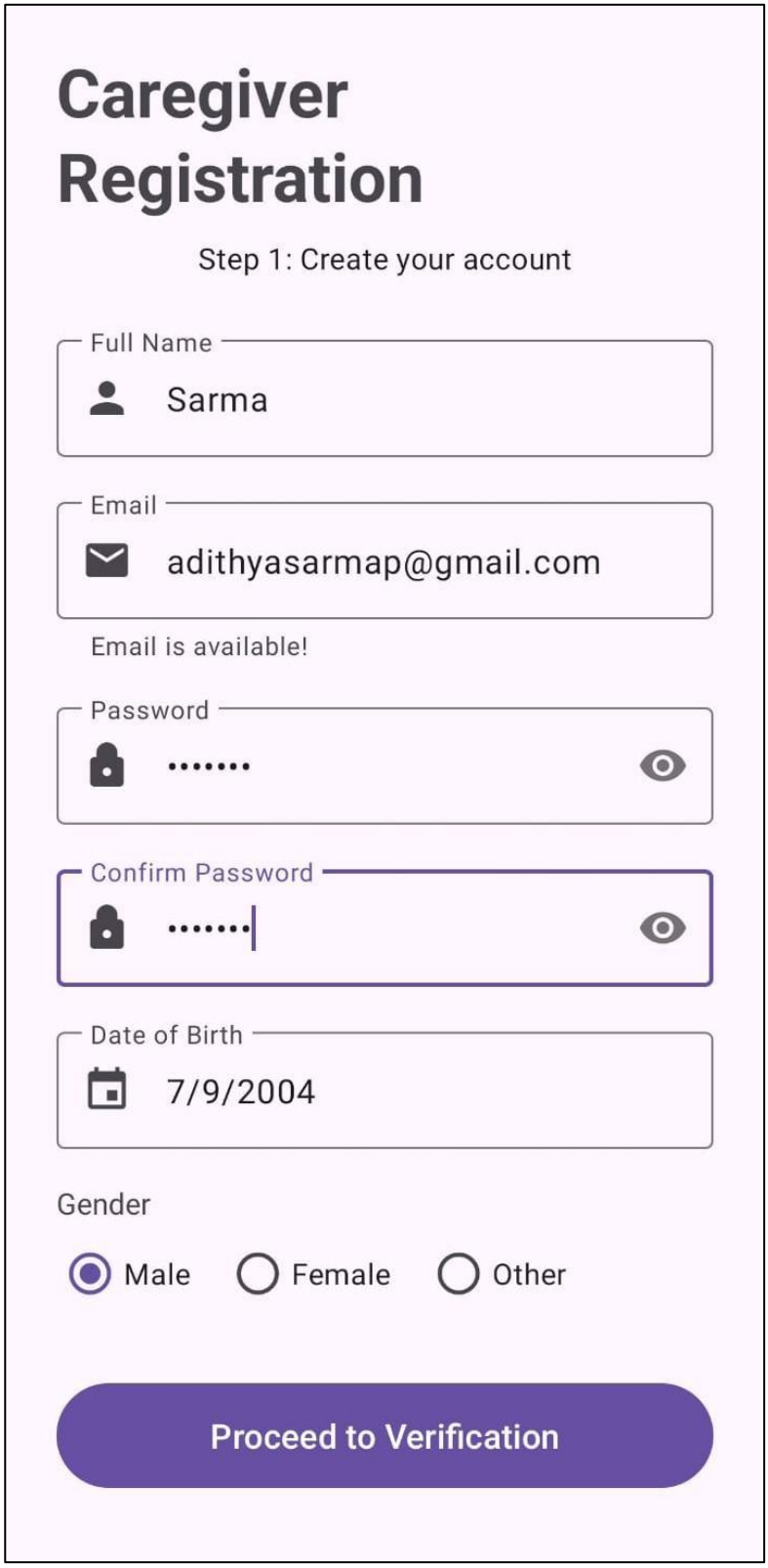


Caregiver Task Assigning

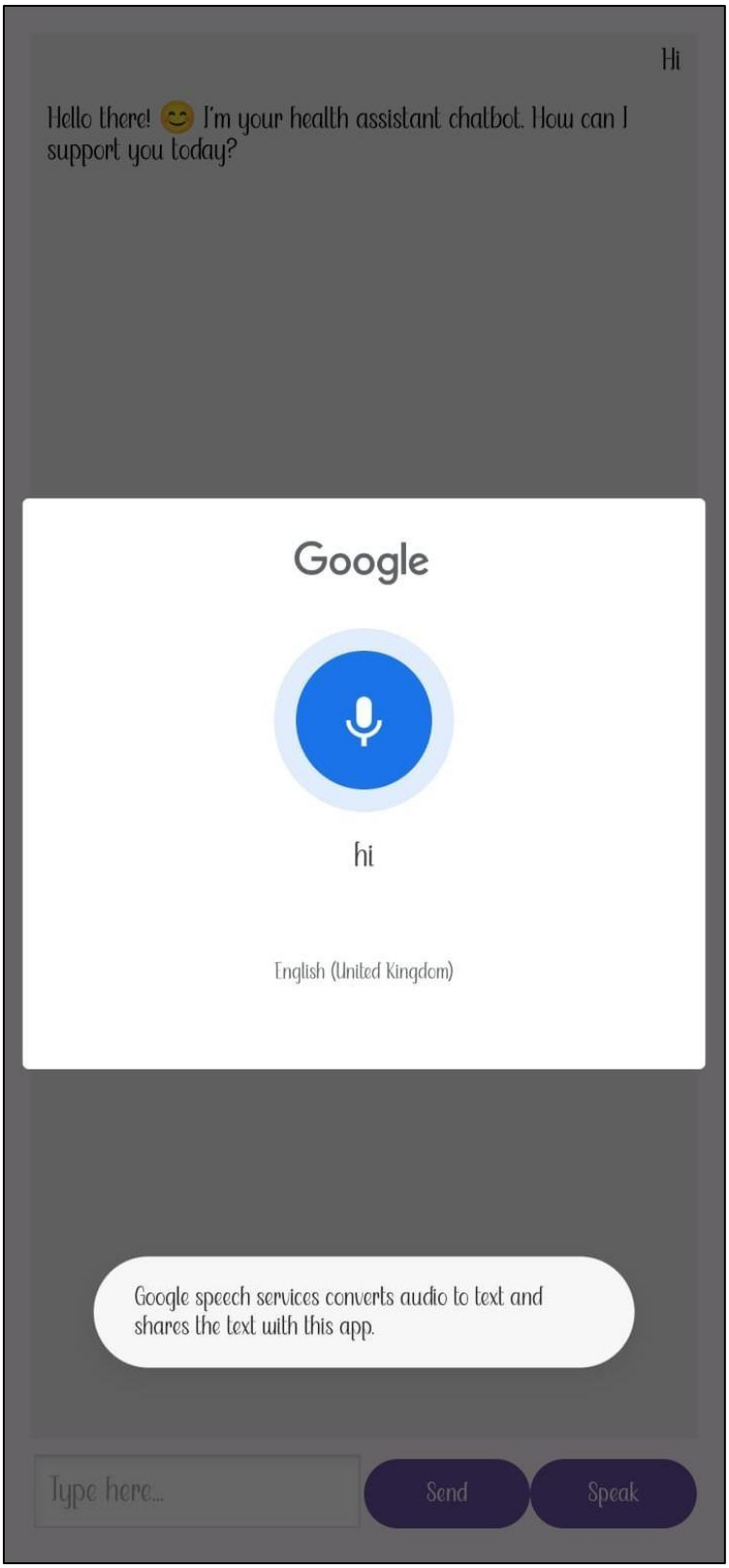


Caregiver task page

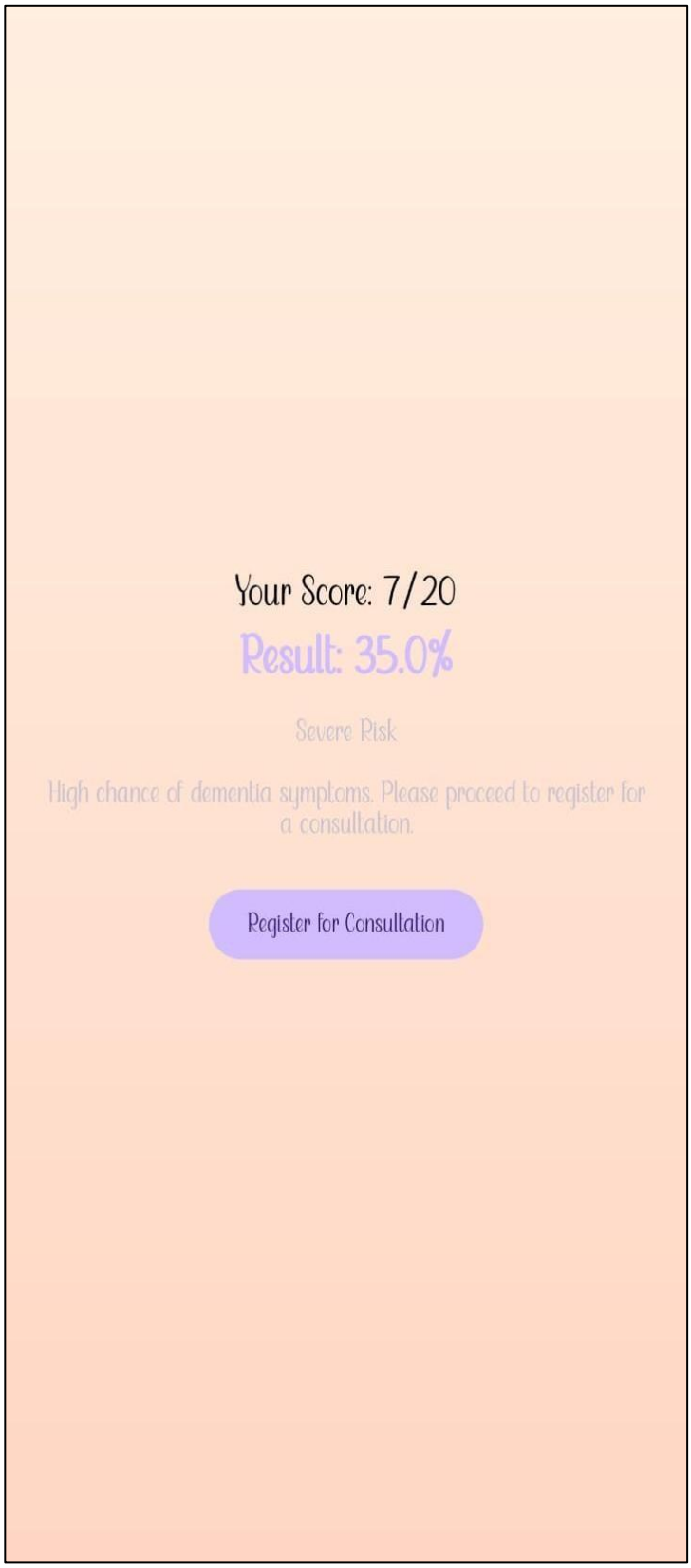
Caregiver registration

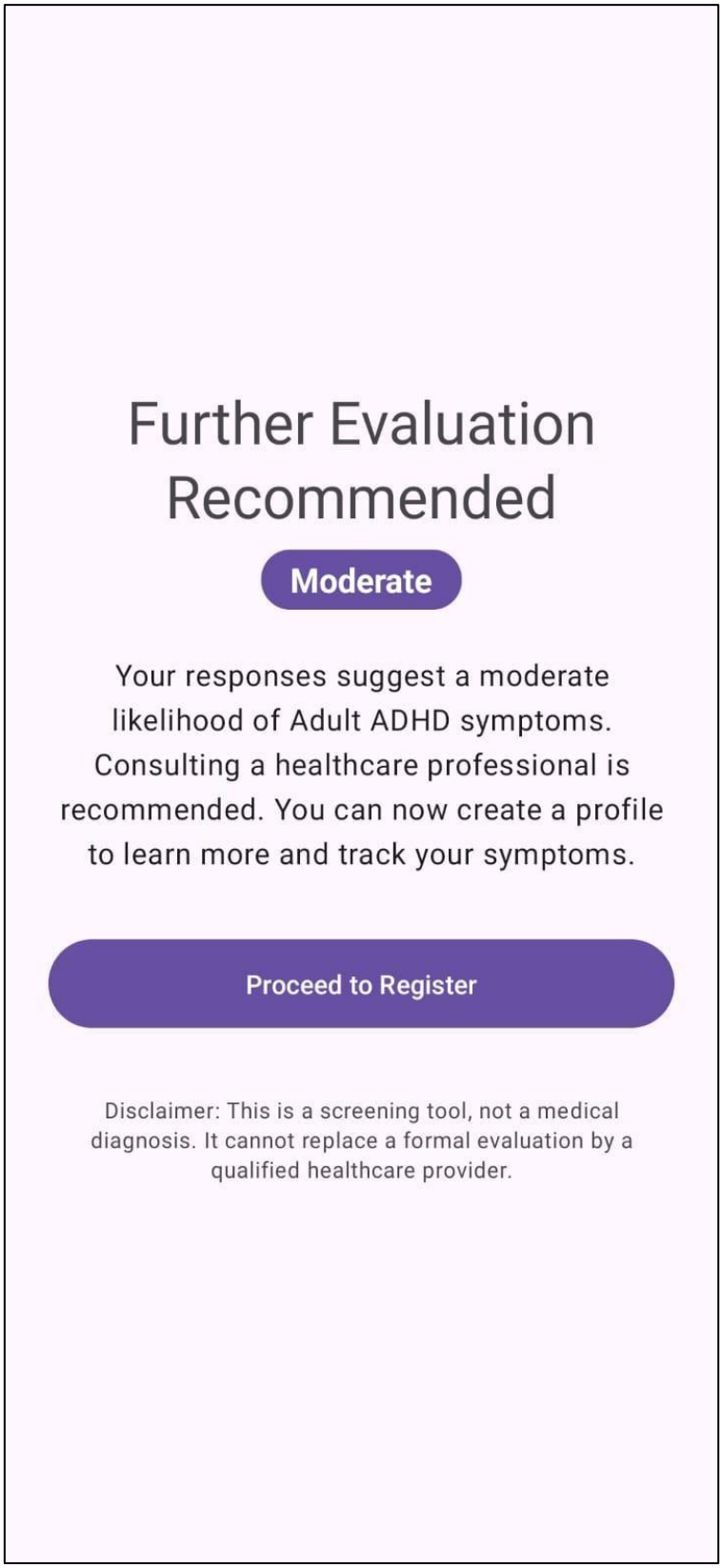


Voice-text chat bot



ADHD and Dementia Result pages





11.2 Sample Code

ADHD Test result

Java_Code

```
package com.example.memora.adhd_test;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import com.example.memora.R;
import com.example.memora.authentication.AdhdRegisterActivity;

public class TestResultActivity extends AppCompatActivity {
@Override
   protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_adhd_test_result);

      TextView tvResultTitle = findViewById(R.id.tvResultTitle);
      TextView tvSeverity = findViewById(R.id.tvSeverity);
      TextView tvResultExplanation = findViewById(R.id.tvResultExplanation);
      TextView tvDisclaimer = findViewById(R.id.tvDisclaimer);
      Button btnProceedToRegister = findViewById(R.id.btnProceedToRegister);
      Button btnExitApp = findViewById(R.id.btnExitApp);

      int[] answers = getIntent().getIntArrayExtra("answers");
      if (answers == null) {
         Toast.makeText(this, "Error: Could not retrieve answers.",
Toast.LENGTH_SHORT).show();
         finish();
         return;
      }
      AsrsScorer.Result result = AsrsScorer.score(answers);
```

```
        tvSeverity.setText(result.severityBand);

        switch (result.severityBand) {
            case "Severe":
                tvResultTitle.setText("Further Evaluation Recommended");
                tvResultExplanation.setText("Your responses indicate a high likelihood of Adult
ADHD symptoms. It is strongly recommended to consult a healthcare professional for a formal
diagnosis. You can now proceed to create a profile to explore coping strategies.");
                btnProceedToRegister.setVisibility(View.VISIBLE);
                btnExitApp.setVisibility(View.GONE);
                break;

            case "Moderate":
                tvResultTitle.setText("Further Evaluation Recommended");
                tvResultExplanation.setText("Your responses suggest a moderate likelihood of Adult
ADHD symptoms. Consulting a healthcare professional is recommended. You can now create a
profile to learn more and track your symptoms.");
                btnProceedToRegister.setVisibility(View.VISIBLE);
                btnExitApp.setVisibility(View.GONE);
                break;

            case "Mild":
                tvResultTitle.setText("Low Likelihood Indicated");
                tvResultExplanation.setText("Your responses suggest some symptoms are present, but
they may not be significant. While a high likelihood of Adult ADHD is not indicated, you can
still speak with a healthcare professional if you have concerns.");
                btnProceedToRegister.setVisibility(View.GONE);
                btnExitApp.setVisibility(View.VISIBLE);
                break;

            case "Minimal":
            default:
                tvResultTitle.setText("Low Likelihood Indicated");
                tvResultExplanation.setText("Your responses do not indicate a high likelihood of Adult
ADHD symptoms based on this screening tool.");
                btnProceedToRegister.setVisibility(View.GONE);
                btnExitApp.setVisibility(View.VISIBLE);
                break;
        }
        tvDisclaimer.setText("Disclaimer: This is a screening tool, not a medical diagnosis. It
cannot replace a formal evaluation by a qualified healthcare provider.");
```

```
        btnProceedToRegister.setOnClickListener(v -> {
            Intent intent = new Intent(TestResultActivity.this, AdhdRegisterActivity.class);
            intent.putExtra("adhdScore", result.totalScore);
            intent.putExtra("adhdSeverity", result.severityBand);
            startActivity(intent);
            finish();
        });
        btnExitApp.setOnClickListener(v -> finishAffinity());
    }
}
```

XML code

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true"
    tools:context=".adhd_test.TestResultActivity">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:orientation="vertical"
        android:padding="24dp">

        <TextView
            android:id="@+id/tvResultTitle"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Screening Result"
            android:textAlignment="center"
            android:textAppearance="@style/TextAppearance.Material3.HeadlineLarge"   />

        <TextView
            android:id="@+id/tvSeverity"
            android:layout_width="wrap_content"
```

```
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:background="@drawable/severity_bond"
            android:paddingStart="16dp"
            android:paddingTop="4dp"
            android:paddingEnd="16dp"
            android:paddingBottom="4dp"
            android:textColor="@android:color/white"
            android:textSize="18sp"
            android:textStyle="bold"
            tools:text="Moderate" />

        <TextView
            android:id="@+id/tvResultExplanation"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="24dp"
            android:lineSpacingMultiplier="1.2"
            android:textAlignment="center"
            android:textAppearance="@style/TextAppearance.Material3.BodyLarge"
            tools:text="Your responses indicate a likelihood of symptoms consistent with Adult
ADHD." />

        <Button
            android:id="@+id/btnProceedToRegister"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="32dp"
            android:padding="14dp"
            android:text="Proceed to Register"
            android:visibility="gone"
            tools:visibility="visible" />

        <Button
            android:id="@+id/btnExitApp"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            android:padding="14dp"
            android:text="Exit App"
            android:visibility="gone"
```

```
        tools:visibility="visible" />

    <TextView
        android:id="@+id/tvDisclaimer"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="32dp"
        android:textAlignment="center"
        android:textAppearance="@style/TextAppearance.Material3.BodySmall"
        android:textColor="?android:attr/textColorSecondary"
        tools:text="Disclaimer: This is a screening tool, not a medical diagnosis." />

  </LinearLayout>
</ScrollView>
```

Caregiver Profile

```
package com.example.memora.task.caregiver_task;

import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;
```

```
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;

import com.example.memora.R;
import com.example.memora.authentication.CaregiverLoginActivity;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.firestore.QueryDocumentSnapshot;

public class CaregiverProfileFragment extends Fragment {

    private static final String TAG = "CaregiverProfileFragment";

    private TextView tvCaregiverName, tvCaregiverEmail, tvCaregiverType;
    private TextView tvPatientName, tvPatientId, tvPatientDob;
    private Button btnLogout;

    private FirebaseFirestore db;
    private FirebaseAuth mAuth;
    private String patientId;
    private String caregiverEmail;

    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup
container, @Nullable Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_caregiver_profile, container, false);

        db = FirebaseFirestore.getInstance();
        mAuth = FirebaseAuth.getInstance();

        if (getArguments() != null) {
            patientId = getArguments().getString("userId");
            caregiverEmail = getArguments().getString("caregiverEmail");
        }

        initializeViews(view);

        if (caregiverEmail != null && !caregiverEmail.isEmpty()) {
            loadCaregiverData();
```

```
        } else {
            Log.e(TAG, "Caregiver email is null. Cannot load profile.");
            if(getContext() != null) Toast.makeText(getContext(), "Error: Caregiver email not
found.", Toast.LENGTH_SHORT).show();
        }

        if (patientId != null && !patientId.isEmpty()) {
            loadPatientData();
        } else {
            Log.e(TAG, "Patient ID is null. Cannot load profile.");
            if(getContext() != null) Toast.makeText(getContext(), "Error: Patient ID not found.",
Toast.LENGTH_SHORT).show();
        }

        btnLogout.setOnClickListener(v -> {
            if (getActivity() == null) return;

            mAuth.signOut();

            Intent intent = new Intent(getActivity(), CaregiverLoginActivity.class);
            intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK |
Intent.FLAG_ACTIVITY_CLEAR_TASK);
            startActivity(intent);
            getActivity().finish();
        });

        return view;
    }

    private void initializeViews(View view) {
        tvCaregiverName = view.findViewById(R.id.tvCaregiverName);
        tvCaregiverEmail = view.findViewById(R.id.tvCaregiverEmail);
        tvCaregiverType = view.findViewById(R.id.tvCaregiverType);
        tvPatientName = view.findViewById(R.id.tvPatientName);
        tvPatientId = view.findViewById(R.id.tvPatientId);
        tvPatientDob = view.findViewById(R.id.tvPatientDob);
        btnLogout = view.findViewById(R.id.btnLogout);
    }

    private void loadCaregiverData() {
        db.collection("Caregivers")
```

```
            .whereEqualTo("email", caregiverEmail)
            .limit(1)
            .get()
            .addOnCompleteListener(task -> {
                if (task.isSuccessful() && !task.getResult().isEmpty()) {
                    QueryDocumentSnapshot document = (QueryDocumentSnapshot)
task.getResult().getDocuments().get(0);

                    tvCaregiverName.setText(document.getString("name"));
                    tvCaregiverEmail.setText(document.getString("email"));

                    tvCaregiverType.setText("Role: " + document.getString("caregiverType"));

                } else {
                    Log.e(TAG, "Error loading caregiver data: ", task.getException());
                    if(getContext() != null) Toast.makeText(getContext(), "Error loading caregiver
data.", Toast.LENGTH_SHORT).show();
                }
            });
    }

    private void loadPatientData() {
        db.collection("Users").document("Patients").collection("DEMENTIA_Patients")
            .document(patientId)
            .get()
            .addOnSuccessListener(document -> {
                if (document.exists()) {
                    tvPatientName.setText(document.getString("name"));
                    tvPatientId.setText("Patient ID: " + document.getString("userId"));
                    tvPatientDob.setText("DOB: " + document.getString("dob"));
                } else {
                    Log.e(TAG, "Patient document does not exist: " + patientId);
                    if(getContext() != null) Toast.makeText(getContext(), "Patient data not found.",
Toast.LENGTH_SHORT).show();
                }
            })
            .addOnFailureListener(e -> {
                Log.e(TAG, "Error loading patient data: ", e);
                if(getContext() != null) Toast.makeText(getContext(), "Error loading patient data.",
Toast.LENGTH_SHORT).show();
            });
```

```
    }
}
```

XML Code

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true"
    tools:context=".task.caregiver_task.CaregiverProfileFragment">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="My Profile"
            android:textAppearance="@style/TextAppearance.Material3.HeadlineMedium"
            android:textStyle="bold" />

        <com.google.android.material.card.MaterialCardView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            app:cardElevation="4dp">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical"
                android:padding="16dp">

                <TextView
                    android:id="@+id/tvCaregiverName"
```

```
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:drawablePadding="8dp"
            android:gravity="center_vertical"
            android:textAppearance="@style/TextAppearance.Material3.TitleLarge"
            app:drawableStartCompat="@drawable/ic_person"
            tools:text="Caregiver Name" />

        <TextView
            android:id="@+id/tvCaregiverEmail"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="12dp"
            android:drawablePadding="8dp"
            android:gravity="center_vertical"
            android:textAppearance="@style/TextAppearance.Material3.BodyLarge"
            app:drawableStartCompat="@drawable/ic_email"
            tools:text="caregiver@gmail.com" />

        <TextView
            android:id="@+id/tvCaregiverType"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="12dp"
            android:drawablePadding="8dp"
            android:gravity="center_vertical"
            android:textAppearance="@style/TextAppearance.Material3.BodyLarge"
            app:drawableStartCompat="@drawable/ic_badge"
            tools:text="Role: Son" />

    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="24dp"
    android:text="Linked Patient"
    android:textAppearance="@style/TextAppearance.Material3.HeadlineMedium"
    android:textStyle="bold" />
```

```
<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    app:cardElevation="4dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <TextView
            android:id="@+id/tvPatientName"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:drawablePadding="8dp"
            android:gravity="center_vertical"
            android:textAppearance="@style/TextAppearance.Material3.TitleLarge"
            app:drawableStartCompat="@drawable/ic_person"
            tools:text="Patient Name" />

        <TextView
            android:id="@+id/tvPatientId"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="12dp"
            android:drawablePadding="8dp"
            android:gravity="center_vertical"
            android:textAppearance="@style/TextAppearance.Material3.BodyLarge"
            app:drawableStartCompat="@drawable/ic_person"
            tools:text="Patient ID: DM_patient_01" />

        <TextView
            android:id="@+id/tvPatientDob"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="12dp"
            android:drawablePadding="8dp"
            android:gravity="center_vertical"
            android:textAppearance="@style/TextAppearance.Material3.BodyLarge"
```

```
            app:drawableStartCompat="@drawable/ic_calendar"
            tools:text="DOB: 01/01/1950" />

        </LinearLayout>
    </com.google.android.material.card.MaterialCardView>

    <Button
        android:id="@+id/btnLogout"
        style="@style/Widget.Material3.Button.OutlinedButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="32dp"
        android:text="Logout" />

</LinearLayout>
</ScrollView>
```

Alarm

Java code

```
package com.example.memora.task;

import android.app.AlarmManager;
import android.app.PendingIntent;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.content.IntentFilter;
import android.os.Build;
import android.os.Bundle;
import android.util.Log;
import android.view.WindowManager;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;
```

```
import androidx.appcompat.app.AppCompatActivity;

import com.example.memora.R;
import com.google.firebase.Timestamp;
import com.google.firebase.firestore.DocumentReference;
import com.google.firebase.firestore.FirebaseFirestore;

import java.text.SimpleDateFormat;
import java.util.HashMap;
import java.util.Locale;
import java.util.Map;

public class AlarmActivity extends AppCompatActivity {

    private String taskName;
    private int taskIdHash;
    private String taskIdString;
    private String patientId;
    private Timestamp dueDate;

    private FirebaseFirestore db;
    private AlarmManager alarmManager;

    private SimpleDateFormat dateDocFormatter;

    private static final String TAG = "ALARM_TEST";

    private CloseActivityReceiver closeActivityReceiver;
    public static final String ACTION_CLOSE =
"com.example.memora.task.CLOSE_ALARM_ACTIVITY";


    @Override
    protected void onCreate(Bundle savedInstanceState) {

        Log.d(TAG, "--- STEP 3: AlarmActivity.onCreate started. UI should be visible. ---");

        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_alarm);
```

```
        db = FirebaseFirestore.getInstance();
        alarmManager = (AlarmManager) getSystemService(Context.ALARM_SERVICE);
        dateDocFormatter = new SimpleDateFormat("yyyy-MM-dd", Locale.US);

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O_MR1) {
            setShowWhenLocked(true);
            setTurnScreenOn(true);
        } else {
            getWindow().addFlags(
                    WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON |

WindowManager.LayoutParams.FLAG_ALLOW_LOCK_WHILE_SCREEN_ON |
                            WindowManager.LayoutParams.FLAG_SHOW_WHEN_LOCKED |
                            WindowManager.LayoutParams.FLAG_TURN_SCREEN_ON
            );
        }
        taskName = getIntent().getStringExtra(TaskAlarmReceiver.TASK_NAME_EXTRA);
        taskIdHash = getIntent().getIntExtra(TaskAlarmReceiver.TASK_ID_HASH_EXTRA, 0);
        taskIdString = getIntent().getStringExtra(TaskAlarmReceiver.TASK_ID_STRING_EXTRA);
        patientId = getIntent().getStringExtra(TaskAlarmReceiver.PATIENT_ID_EXTRA);
        dueDate = getIntent().getParcelableExtra(TaskAlarmReceiver.DUE_DATE_EXTRA);

        Log.d(TAG, "AlarmActivity received Task: " + taskName + " (Firestore ID: " + taskIdString
+ ", ReqCode: " + taskIdHash + ")");

        if (taskIdString == null || patientId == null || dueDate == null || taskIdHash == 0 || taskName
== null) {
            Log.e(TAG, "--- !! ABORTING ALARM ACTIVITY !! Missing essential data (taskName
was null?) ---");
            Toast.makeText(this, "Error: Alarm data missing.", Toast.LENGTH_LONG).show();
            stopAlarmService();
            finish();
            return;
        }
        TextView tvTaskName = findViewById(R.id.tvAlarmTaskName);
        Button btnComplete = findViewById(R.id.btnComplete);

        Button btnDismiss = findViewById(R.id.btnDismiss);

        tvTaskName.setText(taskName);
```

```
        Intent serviceIntent = new Intent(this, RingtonePlayerService.class);
        Log.d(TAG, "--- STEP 4: Attempting to start RingtonePlayerService... ---");
        startService(serviceIntent);

        btnDismiss.setOnClickListener(v -> {
            Log.d(TAG, "Dismiss clicked on Activity page.");

            stopAlarmService();

            cancelAlarms();

            Toast.makeText(this, "Alarm for '" + taskName + "' dismissed.",
Toast.LENGTH_SHORT).show();

            TaskNotificationHelper.cancelNotification(this,
TaskNotificationHelper.ALARM_RINGING_NOTIFICATION_ID);

            finish();
        });

        btnComplete.setOnClickListener(v -> {
            Log.d(TAG, "Complete clicked.");
            stopAlarmService();
            markTaskAsCompleted();
        });
        closeActivityReceiver = new CloseActivityReceiver();
        IntentFilter filter = new IntentFilter(ACTION_CLOSE);
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            registerReceiver(closeActivityReceiver, filter, Context.RECEIVER_NOT_EXPORTED);
        } else {
            registerReceiver(closeActivityReceiver, filter);
        }
        Log.d(TAG, "CloseActivityReceiver registered.");
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        if (closeActivityReceiver != null) {
            unregisterReceiver(closeActivityReceiver);
```

```
            Log.d(TAG, "CloseActivityReceiver unregistered.");
        }
        stopAlarmService();
    }
    private void stopAlarmService() {
        Intent serviceIntent = new Intent(this, RingtonePlayerService.class);
        stopService(serviceIntent);
        Log.d(TAG,"stopAlarmService called.");

    private void markTaskAsCompleted() {
        if (taskIdString == null || patientId == null || db == null || dueDate == null) {
            Log.e(TAG, "Cannot mark task completed: Missing essential data or DB instance.");
            Toast.makeText(this, "Error: Could not update task status.",
Toast.LENGTH_SHORT).show();
            finish();
            return;
        }
        String dateDocId = dateDocFormatter.format(dueDate.toDate());
        DocumentReference taskRef = db.collection("Users").document("Patients")
                .collection("ADHD_Patients").document(patientId)
                .collection("tasks").document(dateDocId)
                .collection("tasks_for_this_date").document(taskIdString);

        Log.d(TAG, "Attempting to mark task complete at path: " + taskRef.getPath());

        Map<String, Object> updates = new HashMap<>();
        updates.put("completed", true);
        updates.put("completedAt", Timestamp.now());

        taskRef.update(updates)
                .addOnSuccessListener(aVoid -> {
                    Log.d(TAG, "Task successfully marked as completed in Firestore.");
                    Toast.makeText(AlarmActivity.this, "'" + taskName + "' marked complete.",
Toast.LENGTH_SHORT).show();

                    cancelAlarms();
                    TaskNotificationHelper.cancelNotification(this,
TaskNotificationHelper.ALARM_RINGING_NOTIFICATION_ID);

                    Task task = new Task(taskIdString, taskName, patientId, dueDate,
Task.PRIORITY_MEDIUM);
```

```
                TaskNotificationHelper.showTaskCompleteNotification(this, task);

                finish();
            })
            .addOnFailureListener(e -> {
                Log.e(TAG, "Error marking task completed in Firestore", e);
                Toast.makeText(AlarmActivity.this, "Failed to update task status.",
Toast.LENGTH_SHORT).show();
                finish();
            });
    }

    private void cancelAlarms() {
        Context context = getApplicationContext();
        if (alarmManager == null || taskIdHash == 0) {
            Log.e(TAG, "Cannot cancel alarm: AlarmManager is null or taskIdHash is 0.");
            return;
        }

        int mainRequestCode = taskIdHash;
        int preAlarmRequestCode = mainRequestCode + 1;

        Intent mainIntent = new Intent(context, TaskAlarmReceiver.class);
        PendingIntent mainPendingIntent = PendingIntent.getBroadcast(
                context,
                mainRequestCode,
                mainIntent,
                PendingIntent.FLAG_NO_CREATE | PendingIntent.FLAG_IMMUTABLE
        );

        if (mainPendingIntent != null) {
            alarmManager.cancel(mainPendingIntent);
            mainPendingIntent.cancel();
            Log.d(TAG, "Main Alarm CANCELED for task '" + taskName + "' (ReqCode: " +
mainRequestCode + ")");
        } else {
            Log.d(TAG, "Main Alarm NOT FOUND for cancel '" + taskName + "' (ReqCode: " +
mainRequestCode + ")");
        }
```

```
        Intent preAlarmIntent = new Intent(context, TaskAlarmReceiver.class);
        PendingIntent preAlarmPendingIntent = PendingIntent.getBroadcast(
                context,
                preAlarmRequestCode,
                preAlarmIntent,
                PendingIntent.FLAG_NO_CREATE | PendingIntent.FLAG_IMMUTABLE
        );
        if (preAlarmPendingIntent != null) {
            alarmManager.cancel(preAlarmPendingIntent);
            preAlarmPendingIntent.cancel();
            Log.d(TAG, "Pre-Alarm CANCELED for task '" + taskName + "' (ReqCode: " +
preAlarmRequestCode + ")");
        } else {
            Log.d(TAG, "Pre-Alarm NOT FOUND for cancel '" + taskName + "' (ReqCode: " +
preAlarmRequestCode + ")");
        }
    }

    private class CloseActivityReceiver extends BroadcastReceiver {
        @Override
        public void onReceive(Context context, Intent intent) {
            String action = intent.getAction();
            if (ACTION_CLOSE.equals(action)) {

                String broadcastTaskId =
intent.getStringExtra(TaskAlarmReceiver.TASK_ID_STRING_EXTRA);

                if (taskIdString != null && taskIdString.equals(broadcastTaskId)) {
                    Log.d(TAG, "CloseActivityReceiver received broadcast. Finishing AlarmActivity.");
                    finish();
                } else {
                    Log.d(TAG, "CloseActivityReceiver ignored broadcast for a different task. (This: " +
taskIdString + " vs Broadcast: " + broadcastTaskId + ")");
                }
            }
        }
    }
}
```

XML code

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#1C1C1E"
    android:padding="32dp"
    tools:context=".task.AlarmActivity">

    <ImageView
        android:id="@+id/ivAlarmIcon"
        android:layout_width="96dp"
        android:layout_height="96dp"
        android:layout_marginBottom="48dp"
        android:src="@android:drawable/ic_lock_idle_alarm"
        android:contentDescription="Alarm bell icon"
        app:tint="#29B6F6"
        app:layout_constraintBottom_toTopOf="@id/tvAlarmTitle"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.3"
        app:layout_constraintVertical_chainStyle="packed" />

    <TextView
        android:id="@+id/tvAlarmTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="TIME TO FOCUS"
        android:textColor="#FFFFFF"
        android:textAppearance="@style/TextAppearance.Material3.DisplayMedium"
        android:textStyle="normal"
        android:letterSpacing="0.1"
        android:alpha="0.85"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/ivAlarmIcon"
        app:layout_constraintBottom_toTopOf="@id/tvAlarmTaskName"
        app:layout_constraintVertical_chainStyle="packed"/>
```

```
<TextView
    android:id="@+id/tvAlarmTaskName"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:paddingHorizontal="16dp"
    android:paddingVertical="12dp"
    android:background="#006064"
    android:text="Task Name Here"
    android:textColor="#FFFFFF"
    android:textAppearance="@style/TextAppearance.Material3.HeadlineSmall"
    android:textAlignment="center"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/tvAlarmTitle"
    app:layout_constraintBottom_toTopOf="@id/buttons_container"
    app:layout_constraintWidth_percent="0.9"
    app:layout_constraintVertical_bias="0.2"/>
<androidx.constraintlayout.widget.ConstraintLayout
    android:id="@+id/buttons_container"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:paddingTop="48dp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent">
    <Button
        android:id="@+id/btnComplete"
        style="@style/Widget.Material3.Button.ElevatedButton"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="COMPLETED"
        android:textSize="18sp"
        android:paddingVertical="18dp"
        android:backgroundTint="#006064"
        android:textColor="#FFFFFF"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:cornerRadius="24dp" />
    <Button
```

```
        android:id="@+id/btnDismiss"
        style="@style/Widget.Material3.Button.OutlinedButton"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="DISMISS"
        android:textColor="#FFFFFF"
        android:textSize="18sp"
        android:paddingVertical="18dp"
        app:strokeColor="#FFFFFF"
        app:cornerRadius="24dp"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/btnComplete" />
    </androidx.constraintlayout.widget.ConstraintLayout>

</androidx.constraintlayout.widget.ConstraintLayout>
```

Caregiver notes

Java code

```
package com.example.memora.notes;

import android.os.Bundle;
import android.util.Log;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import com.example.memora.R;
import com.google.firebase.firestore.CollectionReference;
import com.google.firebase.firestore.FieldValue;
import com.google.firebase.firestore.FirebaseFirestore;

import java.util.HashMap;
import java.util.Map;
```

```
public class AddNewNoteActivity extends AppCompatActivity {

    private static final String TAG = "AddNewNoteActivity";

    private EditText etNoteContent;
    private Button btnSaveNote;
    private TextView tvClientAssignment;
    private CaregiverNoteModel noteToEdit = null;

    private FirebaseFirestore db;

    private String currentPatientId;
    private String caregiverAuthId;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_add_new_note);
        db = FirebaseFirestore.getInstance();

        currentPatientId = getIntent().getStringExtra("userId");
        caregiverAuthId = getIntent().getStringExtra("caregiverAuthId");
        initializeViews();

        if (currentPatientId != null && !currentPatientId.isEmpty() && caregiverAuthId != null &&
!caregiverAuthId.isEmpty()) {
            tvClientAssignment.setText("Patient ID: " + currentPatientId);
            Log.d(TAG, "Received patient ID: " + currentPatientId + " and caregiverAuthId: " +
caregiverAuthId);
        } else {
            Log.e(TAG, "CRITICAL: Patient ID or Caregiver ID is missing in Intent.");
            Toast.makeText(this, "Error: User data is missing. Cannot save note.",
Toast.LENGTH_LONG).show();
            finish();
            return;
        }

        // Check for Edit Mode
        if (getIntent().getSerializableExtra("NOTE_TO_EDIT") != null) {
            noteToEdit = (CaregiverNoteModel)
getIntent().getSerializableExtra("NOTE_TO_EDIT");
```

```
            if (getSupportActionBar() != null) getSupportActionBar().setTitle("Edit Observation");
            btnSaveNote.setText("Update Note");
            etNoteContent.setText(noteToEdit.getContent());
            etNoteContent.setSelection(etNoteContent.getText().length());
        } else {
            if (getSupportActionBar() != null) getSupportActionBar().setTitle("Record New
Observation");
        }

        btnSaveNote.setOnClickListener(v -> saveNote());
    }
    private void initializeViews() {
        etNoteContent = findViewById(R.id.etNoteContent);
        btnSaveNote = findViewById(R.id.btnSaveNote);
        tvClientAssignment = findViewById(R.id.tvClientAssignment);
    }
    private void saveNote() {
        String content = etNoteContent.getText().toString().trim();

        if (content.isEmpty()) {
            Toast.makeText(this, "Note content cannot be empty.", Toast.LENGTH_SHORT).show();
            return;
        }
        if (currentPatientId == null || currentPatientId.isEmpty() || caregiverAuthId == null ||
caregiverAuthId.isEmpty()) {
            Toast.makeText(this, "Error: User data is missing. Cannot save.",
Toast.LENGTH_SHORT).show();
            return;
        }

        CollectionReference notesCollection = db.collection("Users").document("Caregivers")
                .collection("Caregiver_Profiles").document(caregiverAuthId)
                .collection("CaregiverNotes");


        if (noteToEdit == null) {
            CaregiverNoteModel newNote = new CaregiverNoteModel(
                    currentPatientId,
                    "N/A",
                    content
            );
```

```
            notesCollection.add(newNote)
                    .addOnSuccessListener(documentReference -> {
                        Toast.makeText(this, "Note saved successfully!",
Toast.LENGTH_SHORT).show();
                        finish();
                    })
                    .addOnFailureListener(e -> {
                        Log.e(TAG, "Error saving note", e);
                        Toast.makeText(this, "Error saving note: " + e.getMessage(),
Toast.LENGTH_SHORT).show();
                    });
            } else {
                if (noteToEdit.getNoteId() == null) {
                    Toast.makeText(this, "Error: Cannot update note without ID.",
Toast.LENGTH_SHORT).show();
                    return;
                }

                Map<String, Object> updates = new HashMap<>();
                updates.put("content", content);
                updates.put("timestamp", FieldValue.serverTimestamp());

                notesCollection.document(noteToEdit.getNoteId())
                        .update(updates)
                        .addOnSuccessListener(aVoid -> {
                            Toast.makeText(this, "Note updated successfully!",
Toast.LENGTH_SHORT).show();
                            finish();
                        })
                        .addOnFailureListener(e -> {
                            Log.e(TAG, "Error updating note", e);
                            Toast.makeText(this, "Error updating note: " + e.getMessage(),
Toast.LENGTH_SHORT).show();
                        });
            }
        }
    }
```